



Infor Mongoose Integrating IDOs with External Applications

Important Notices

The material contained in this publication (including any supplementary information) constitutes and contains confidential and proprietary information of Infor.

By gaining access to the attached, you acknowledge and agree that the material (including any modification, translation or adaptation of the material) and all copyright, trade secrets and all other right, title and interest therein, are the sole property of Infor and that you shall not gain right, title or interest in the material (including any modification, translation or adaptation of the material) by virtue of your review thereof other than the non-exclusive right to use the material solely in connection with and the furtherance of your license and use of software made available to your company from Infor pursuant to a separate agreement, the terms of which separate agreement shall govern your use of this material and all supplemental related materials ("Purpose").

In addition, by accessing the enclosed material, you acknowledge and agree that you are required to maintain such material in strict confidence and that your use of such material is limited to the Purpose described above. Although Infor has taken due care to ensure that the material included in this publication is accurate and complete, Infor cannot warrant that the information contained in this publication is complete, does not contain typographical or other errors, or will meet your specific requirements.

As such, Infor does not assume and hereby disclaims all liability, consequential or otherwise, for any loss or damage to any person or entity which is caused by or relates to errors or omissions in this publication (including any supplementary information), whether such errors or omissions result from negligence, accident or any other cause.

Without limitation, U.S. export control laws and other applicable export and import laws govern your use of this material and you will neither export or re-export, directly or indirectly, this material nor any related materials or supplemental information in violation of such laws, or use such materials for any purpose prohibited by such laws.

Trademark Acknowledgements

The word and design marks set forth herein are trademarks and/or registered trademarks of Infor and/or related affiliates and subsidiaries. All rights reserved. All other company, product, trade or service names referenced may be registered trademarks or trademarks of their respective owners.

Publication Information

Release: Infor Mongoose 9.00

Publication date: November 15, 2013

Contents

About This Guide	5
Intended audience	5
Related documents	5
Contacting Infor	6
Chapter 1: Overview	9
About IDOs	9
IDO Runtime Service	10
IDO requests and responses	10
Types of requests/responses	10
Structure of requests	11
Structure of responses	12
Protocols	12
XML request/response documents	12
IDO .NET class libraries, command interface, and extension classes	13
Application Event System	15
.NET Web service calls	15
SharePoint integration	16
RSDAO calls	16
Integrating directly with the database (dynamic IDOs)	16
Standard versus dynamic IDOs	17
Directly accessing the database programmatically	17
Architecture of IDO communications	18
Client application communication with IDOs	18
Framework services communication with IDOs	19
IDO request/response communication with IDOs	20
IDO extension class communication with IDOs	21
User middleware permissions and licensing	21
Setting permissions for an IDO “user”	21

Licensing	22
For more information	22
Chapter 2: Using XML IDO Request and Response Documents	25
About XML IDO requests and responses	25
Replicating XML documents	26
Posting XML documents	26
Request documents	27
Response documents	27
XML request/response header information	28
Request and response types	29
GetConfigurations	29
OpenSession	34
GetPropertyInfo	38
LoadCollection	44
UpdateCollection	51
Invoke	61
CloseSession	63
Passing dates to the IDO Runtime Service	64
Calling dynamic IDOs in XML documents	64
Chapter 3: Mongoose-ION Integration	63
Overview: Mongoose, ION, and BODs	64
Mongoose-ION exchanges	64
Information needed for the integration	65
Prerequisites	65
Setting up Mongoose for ION	65
Setting up a configuration that matches your local site	66
Setting up the logical ID and BOD replication	66
Setting up information that will flow to the other application	68
Verifying that BODs are replicating	68
Initial load: Running the Replication Document Manual Request Utility	68
Setting up ION for Mongoose	69
About the Mongoose connection point	69
Activating the flow	69
Testing the integration	69
Troubleshooting and maintenance	70

Error in Replication Document Manual Request Utility – timeout.	70
Data is not flowing properly between the systems	70
Obsolete BODs – deleting	71
Drillbacks	71
Chapter 4: Using the Application Event System with IDOs	75
Event action types available for IDO integration.	75
Example: Opening a session and requesting data for a remote site	76
For more information	80
Chapter 5: Using the .NET Web Service API	81
About .NET Web service API calls	81
Prerequisite	82
Creating API calls	82
GetConfigurationNames.	82
CreateSessionToken	83
LoadDataSet	84
SaveDataSet	85
CallMethod	86
.NET Web service example	88
Calling dynamic IDOs using .NET	89
Infor .NET Web service test utility	90
Chapter 6: Using Infor SharePoint Web Parts	91
About the IDO Web Parts	91
Prerequisites.	92
Deploying the Web Parts in SharePoint	92
Configuring the connection to Mongoose at SharePoint sites	93
Adding the Web Parts to a SharePoint page	95
Configuring the Web Parts to display Mongoose data in SharePoint.	96
Infor Chart Web Part.	96
Infor List Web Part	97
Exporting Web Parts.	98
Importing Web Parts.	99
Drilling back to Mongoose application forms	99

Chapter 7: Directly Accessing the Database Programmatically	101
InitSessionContextSp.	101
CloseSessionContextSp.	102
Index	103

About This Guide

This guide describes what Infor Mongoose IDOs (Intelligent Data Objects) are and how to use them to integrate with external applications. Information is provided on how to:

- Construct requests and responses for IDOs
- Integrate Mongoose-based applications with other ION-enabled applications
- Use the Application Event System to use IDOs
- Use the .NET Web service API with IDOs
- Integrate Infor Framework IDOs with SharePoint sites

Intended audience

The primary audience for this guide includes Mongoose form and application developers tasked with creating or modifying forms that use IDOs. Secondary audiences include support, marketing, and administrative personnel tasked with supporting Mongoose application development.

Related documents

You can find the documents in the product documentation section of the Infor Xtreme Support portal, as described in "Contacting Infor" on page 6. "Contacting Infor" on page 6

- *Infor Mongoose IDO Development Guide* – Provides information on how to develop your own IDOs based on the Infor Mongoose framework and how to extend the functionality of an existing IDO by adding methods and event handlers. This guide describes the classes and libraries available to you, and provides some examples of their use.
- *Infor Mongoose Design Considerations* – Includes a list and descriptions of the tools available for creating and editing forms, IDOs, events, and event handlers. It also describes the administrative and development utilities to use for managing a development project and deploying an application.

This guide also includes information you might need about external touch points, application messages, licensing for IDOs, and so on. We strongly recommend that you read through this guide before creating or modifying IDOs.

-
- *Guide to the Application Event System* – Describes the Mongoose Application Event System, which developers and system administrators can use to create events and event handlers that automate tasks. It also contains sample scenarios that will help you understand how you can take advantage of the Application Event System.

Contacting Infor

If you have questions about Infor products, go to the Infor Xtreme Support portal at:

<http://www.infor.com/inforxtreme>

If you have questions specific to Infor Mongoose, go to the Infor Mongoose product portal at:

<https://mongoose.infor.com/portal/>

If we update this document after the product release, we will post the new version on these web sites. We recommend that you check these web sites periodically for updated documentation.

If you have comments about Infor documentation, contact documentation@infor.com.

An *Intelligent Data Object* (IDO) is a business software object that encapsulates units of information and logic that are called from the client layer to interact with data in the database. This chapter provides an overview of what IDOs are and how they work. We include general information about:

- The Infor Mongoose services used to access and manipulate data within IDOs ("IDO Runtime Service" on page 10)
- How requests for data and their responses are structured ("IDO requests and responses" on page 10)
- The protocols that can be used to interact with IDOs ("Protocols" on page 12)
- How to integrate with IDOs from an external application ("Integrating directly with the database (dynamic IDOs)" on page 16)
- How communications between IDOs and external applications are structured ("Architecture of IDO communications" on page 18)
- The requirements for accessing IDOs from external applications ("User middleware permissions and licensing" on page 21)

About IDOs

The IDO's job is to transport collections of data back and forth, with any validation or rules needed, between the client and the database.

For example, the UserNames IDO handles the data and logic shown on many of the forms that deal with system users (user ID, password, email address, permissions, and so on).

An IDO consists of the following elements:

- A set of *properties* - A property can represent persistent data stored in the application database, derived data, or temporary data used to communicate information to the middleware. A property can also represent a whole subcollection of data.
- A set of *standard methods* - All IDOs implement these methods:
 - **LoadCollection** retrieves a collection of rows from the database.
 - **UpdateCollection** takes a set of rows marked for insert, update, or delete, and executes the appropriate SQL code against the database.
 - **GetPropertyInfo** returns detailed information about the properties supported by the IDO.

- **Invoke** allows you to execute a custom method.

IDO definitions are stored as metadata in an objects database and are accessed through IDO editing forms. These forms are available to users with a developers' license.

Through configurations, application databases are linked with an objects database and a forms database.

IDO Runtime Service

Most access to the application database is through the IDO runtime service. The primary responsibilities of the IDO runtime service include querying data sets, saving data, and calling methods. In IDO terms, as mentioned previously, these actions are known as LoadCollection, UpdateCollection, GetPropertyInfo, and Invoke.

IDO requests and responses

Interactions with the IDO runtime are based on requests and responses. The caller builds an IDO request and sends it to the IDO runtime engine to be executed. The IDO runtime engine builds and returns a response to the caller containing the results of the requested action.

Types of requests/responses

Actions that can be requested include:

- **Open session** – When provided with a user ID, password, and the name of the configuration to access, the system uses this information to select the utility server and site database to use for the session, and returns a session ID. This is essentially the same as a user logging into the application; it opens a session where the caller can communicate with the IDO.
- **Get configurations** – Gets a list of the application configuration names available on a server and the relevant runtime configuration information associated with each. Use this action only if you do not already know the configuration or URL you want to use.
- **Load collection** – Queries for the contents of either an IDO collection or a database table.
- **Get property information** – Gets a list of the properties and attributes of a collection, or the columns in a database table.
- **Update collection** – Inserts, adds, or deletes records from a collection or a table.
- **Invoke method** – Executes the IDO method. This method can be code in a custom assembly, or it can be a stored procedure.
- **Close session** – Closes the session. This action is only required when accessing IDOs from an external application.

Structure of requests

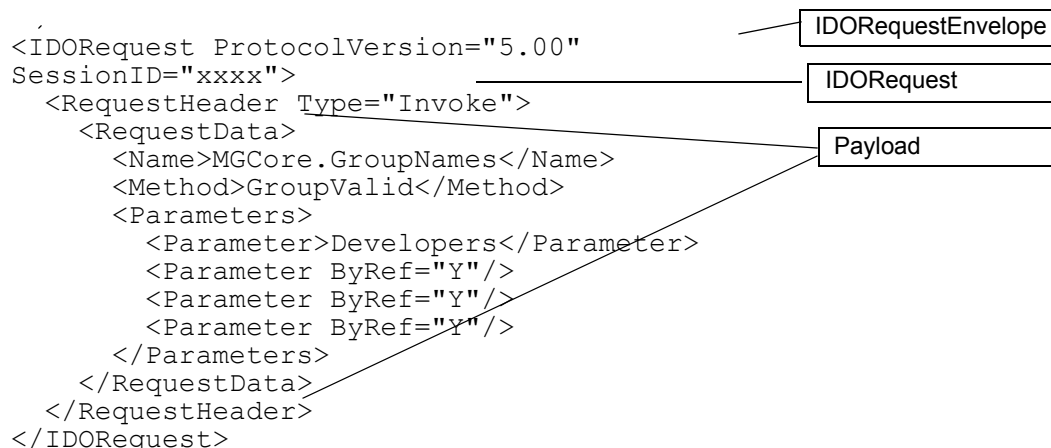
IDO requests have the following hierarchical structure:

- **IDORequestEnvelope** (contains one or more IDORequests)
 - **IDO Request** (contains zero or one request payloads)
 - **Request Payload** (contains data required to complete the request)

The IDORequestEnvelope is the top-level container for a collection of IDORequests. It also contains header information such as the caller's Session ID.

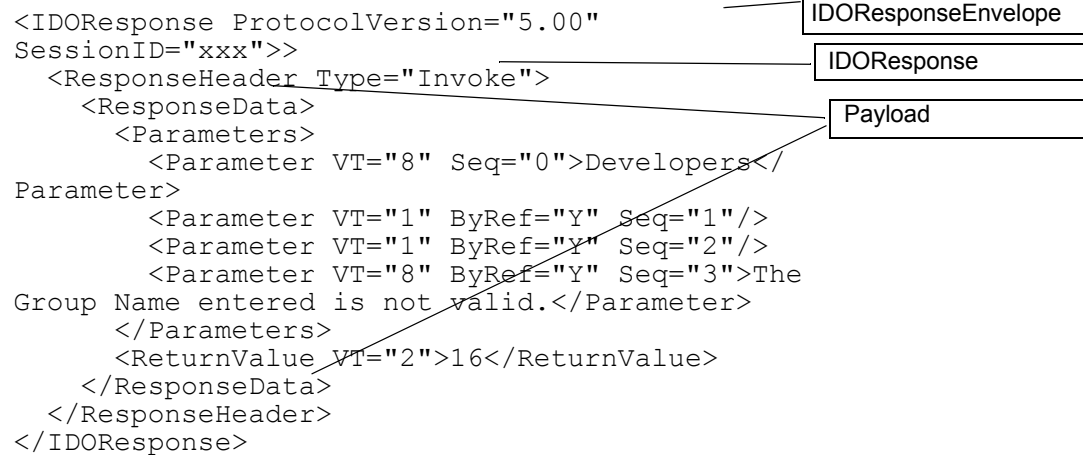
Each IDORequest has a request type (OpenSession, LoadCollection, Invoke, etc.). It also contains an optional request payload that is dependent on the request type. For example, the payload for an OpenSession request contains login information for a user (user ID, password, and configuration).

This example shows the structure of an Invoke IDO request in XML format. This example asks the IDO runtime to validate whether "Developers" is a defined Group name.



Structure of responses

Responses from the IDO runtime have a parallel hierarchy corresponding to the contents of the IDO request. The payload includes the requested data or return values. The response to our example request might be something like this:



TIP: A good way to become more familiar with XML IDO request and response document structure is to view the documents logged by the "Wire Tap" feature of the IDO Runtime Development Server. When the wire tap is turned on, everything a user does in an application session is recorded as a series of request/response documents.

Protocols

An application can use any of the following protocols to make IDO requests and receive responses.

XML request/response documents

At the lowest level, interactions between callers and the IDO runtime are based on XML documents like the ones in the examples on the previous pages. The use, syntax, and examples of these XML documents are described in Chapter 2, "Using XML IDO Request and Response Documents."

Generally XML documents are used for synchronizing data (through non-transactional replication) with another application that expects XML import/exports. This applies to the external financials interface and applications interfacing through Infor ION.

IDO .NET class libraries, command interface, and extension classes

You can also programmatically access the IDO runtime using the Mongoose.IDO.Client class library or the IDO .NET API. The Command Interface (IIDOCCommands) can be used by standalone client applications, by WinStudio scripts, by user controls, and within IDO extension classes.

Mongoose.IDO.Client implements this interface; it supports all IIDOCCommands methods (LoadCollection, UpdateCollection, GetPropertyInfo, and Invoke), plus additional methods that are necessary to create a standalone application but which should not be called from a WinStudio form script or an IDO extension class (OpenSession, CloseSession, GetConfigurations).

Protocol class library

Mongoose provides a class library (IDOProtocol) with classes that correspond to each level in the hierarchy of an IDO request and response. Thus a developer rarely, if ever, has to work directly with the XML. The top level of the hierarchy is implemented by the IDORequestEnvelope class. The IDO request is implemented by the IDORequest class. The request payload implementation, if any, varies depending on the request type. For example, an Invoke request is implemented by the InvokeRequestData class, and other similar classes are available for each of the request types that carry a payload. The protocol classes provide a ToXML method and a FromXML method to easily convert to and from XML text and protocol classes.

This example shows the same Invoke request as in the previous example, but using code that includes an IDO protocol class. Compare the code to the XML.

<pre><IDORequest ProtocolVersion="5.00" SessionID="xxxx"> <RequestHeader Type="Invoke"> <RequestData> <Name>MGCore.GroupNames</Name> <Method>GroupValid</Method> <Parameters> <Parameter>Developers</Param <Parameter ByRef="Y"/> <Parameter ByRef="Y"/> <Parameter ByRef="Y"/> </Parameters> </RequestData> </RequestHeader> </IDORequest></pre>	<pre>IDORequestEnvelope env = new IDORequestEnvelope(); InvokeRequestData data = new InvokeRequestData(); data.IDOName = "MGCore.GroupNames"; data.MethodName = "GroupValid"; data.Parameters.Add("Developers"); data.Parameters [1].ByRef = true; // ...repeat for parameters 3 and 4... env.Requests.Add(new IDORequest(RequestType.Invoke, data); Console.WriteLine(env.ToXML());</pre>
--	--

IDO responses are implemented in classes that mirror the requests, with the IDOResponseEnvelope at the top of the hierarchy.

IDO command interface

A much simpler programmatic interface is also available: the IDO command interface (IIDOCommand), which provides wrapper methods that typically reduce the interaction to a single line of code. This example shows the same Invoke request as in the previous example, this time using the IDO command interface.

<pre><IDORequest ProtocolVersion="5.00" SessionID="xxxx"> <RequestHeader Type="Invoke"> <RequestData> <Name>MGCore.GroupNames</Name> <Method>GroupValid</Method> <Parameters> <Parameter>Developers</Parameter> <Parameter ByRef="Y"/> <Parameter ByRef="Y"/> <Parameter ByRef="Y"/> </Parameters> </RequestData> </RequestHeader> </IDORequest></pre>	<pre>' invoke a method Dim resp As InvokeResponseData resp = Context.Commands.Invoke("MGCore.GroupNames", "GroupValid", "Developers", IDONull.Value, IDONull.Value, IDONull.Value,) group = resp.Parameters(1).Value</pre>
--	--

Although using IDO command interface wrappers in your code is easier than using the IDO protocol classes, the interface is limited to one response per command, and generally supports only the most common parameters for each type of command. Using the IDO Protocol class library is more flexible: you can build multiple requests in one "envelope" and you can build hierarchical LoadCollection or UpdateCollection requests.

Client class library

The IDORequestClient .NET class library has one class, Client. When you create an instance of the Client class, it publishes the IDO access commands (LoadCollection, UpdateCollection, GetPropertyInfo, and Invoke), plus additional commands (OpenSession, GetConfiguration, CloseSession) that are only used for external application communications.

IDO extension classes

An IDO extension class is a .NET class that allows developers to extend the functionality of an IDO by adding methods and event handlers. IDO extension classes are compiled into a .NET class library assembly and stored in the objects (IDO metadata) database. The IDO runtime engine loads these assemblies on demand and calls methods and event handlers in the extension classes in response to IDO requests.

AppDB class

The AppDB class provides direct access to the application database, plus access to common MongoDB features such as application messages and session variables. AppDB is a class that can be used only from IDO extension classes. It is defined in the IDOCORE class library.

For more information about the IDO command interface, class libraries, and extension classes, see the *IDO Development Guide*.

Application Event System

You can use Application Event System forms to set up application event handlers that communicate with IDOs. For example, to invoke the existing method `GetSystemParms` in the `SLInvparms` IDO, set up an event action of type `Call IDO Method` with these parameters:

```
IDO("SLInvparms")  
METHOD("GetSystemParm")  
PARMS( RE(ParmValue), E(ParmName) )
```

For more information on how to set up events for IDO communication, see Chapter 4, "Using the Application Event System with IDOs."

.NET Web service calls

It is possible to use VB .NET, C#, or any .NET CLR-compatible language to make calls over the Internet that load or update IDO collections, or run methods. The .NET Web service is another layer on top of the Infor Web service (XML) layer.

This sample C# code uses the Infor MongoDB IDO Web Service to return a dataset that retrieves data from an IDO named `MCB_Metals10`. This code uses a session token to communicate with the IDO, passing along:

- The IDO name (**MCB_Metals10**)
- A comma-delimited list of properties to return (**Name, ChemicalFormula**)
- The specifications for filtering, ordering, and posting (all blank, in this case)
- The max number of records to return (**0**, which in this case means "no limit")

```
IDOWebService.IDOWebService idoWS = new IDOWebService.IDOWebService();
String sessionToken = idoWS.CreateSessionToken(
    "mbair1",
    "",
    "MG90Local" );
DataSet customers = idoWS.LoadDataSet(
    sessionToken,
    "MCB_Metals10",
    "Name, ChemicalFormula",
    "",
    "",
    "",
    0 );
```

For more information, see Chapter 5, "Using the .NET Web Service API."

SharePoint integration

Infor SharePoint Web Parts retrieve and display on a SharePoint site the IDO data from a specific application configuration. The Web Parts display application information as charts or tabular lists, based on the specified IDO, properties, and filters.

For more information, see Chapter 6, "Using Infor SharePoint Web Parts."

RSDAO calls

RSDAO is provided for backward compatibility but is not generally used in the current version, since the `Mongoose.IDO.Client` class library replaces the RSDAO COM library.

RSDAO is a set of components that implements a small set of objects that wrap the functionality provided by the IDOs. Using RSDAO, the programmer can create instances of any IDO, retrieve data, update data, and call methods of the IDO. Since RSDAO allows users to directly access IDOs, the system checks the license on those requested IDOs. The IDOs must be properly licensed before access is allowed. For more information about accessing IDOs through RSDAO, see the RSDAO help file, available on the Infor Xtreme Support portal (see "Contacting Infor" on page 6).

Integrating directly with the database (dynamic IDOs)

IDO `LoadCollection` and `UpdateCollection` requests can directly query or update the SQL database tables through "dynamic IDOs." Using dynamic IDOs, you can also directly invoke stored procedures from the SQL database, or from custom events defined for forms. For details on how to do this, see the *Infor Mongoose Design Considerations* guide. Use the standard request syntax, except:

- For collection requests, instead of the collection name, specify the table name (in the format `TABLE!table_name`).
- When invoking a stored procedure, instead of the IDO name, specify **SP!** and instead of the method name, specify the stored procedure name. For example:

```
Imports Mongoose.IDO
Imports Mongoose.IDO.Protocol
Public Class SPExample
    Shared Sub Main()
        Using client As Client = New Client()
            Dim openResponse As OpenSessionResponseData
            openResponse = client.OpenSession("user", "password", "configuration")
            If openResponse.LogonSucceeded Then
                Dim invokeResponse As InvokeResponseData
                invokeResponse = client.Invoke("SP!", "MyStoredProc", "parm1", "parm2")
            End If
        End Using
    End Sub
End Class
```

Standard versus dynamic IDOs

Standard and dynamic IDOs behave exactly the same at runtime. However, a dynamic IDO simply binds to a table or a view and publishes each column as a property. In contrast, standard IDOs are defined in the objects database (IDO metadata) and can support these additional features:

- A custom load method can be used to load a collection.
- The IDO can publish data from multiple database tables.
- A property name can be different from the name of the column it is bound to. In Mongoose-based applications, this is almost always the case.
- Properties can be calculated values that are not represented by a column in the database.
- The IDO can publish non-persistent properties (often used for passing user input as parameters to a process).
- The IDO can publish subcollection properties (necessary to use WinStudio's subcollection features).

Directly accessing the database programmatically

You can programmatically access the application database directly (without going through the IDO layer) using a development tool such as VB or C#.

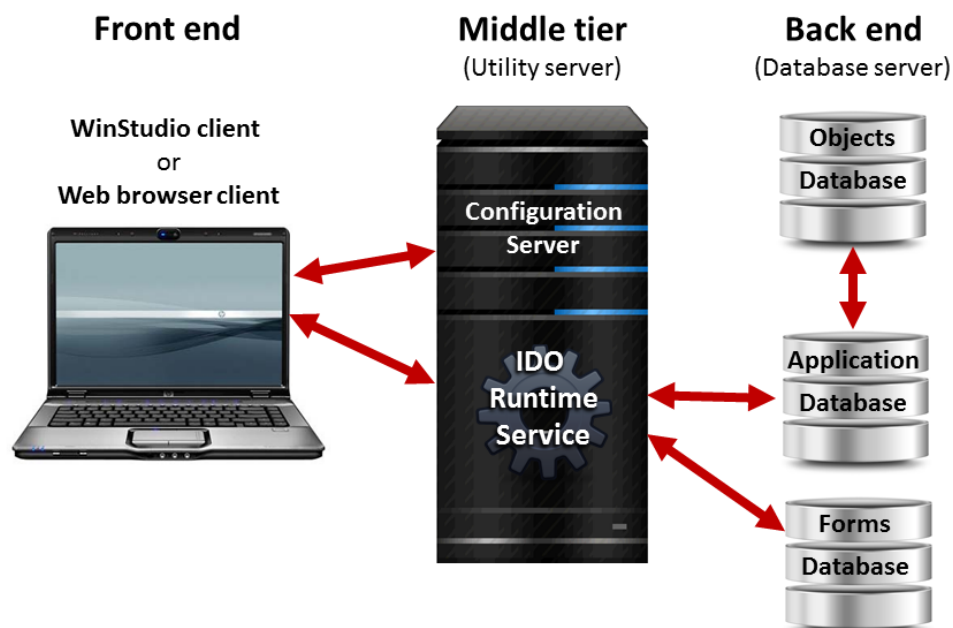
The IDO layer usually handles the session management for connections. So, if your program is bypassing the IDO layer, it must run procedures to perform session management. This is described more fully in Chapter 7, “Directly Accessing the Database Programmatically.”

Architecture of IDO communications

This section shows how the client, Mongoose services, IDO request/response calls, and IDO extension classes communicate with the IDO runtime and the back-end databases.

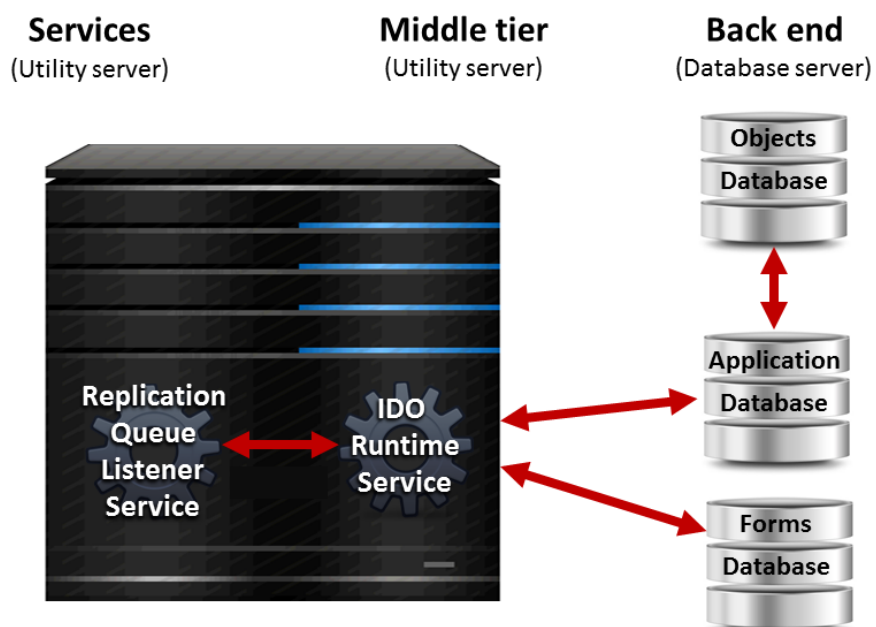
Client application communication with IDOs

When a user logs into a WinStudio (application) session on a client, the system sends a "Get Configurations" request to the Config Server page on a utility server.



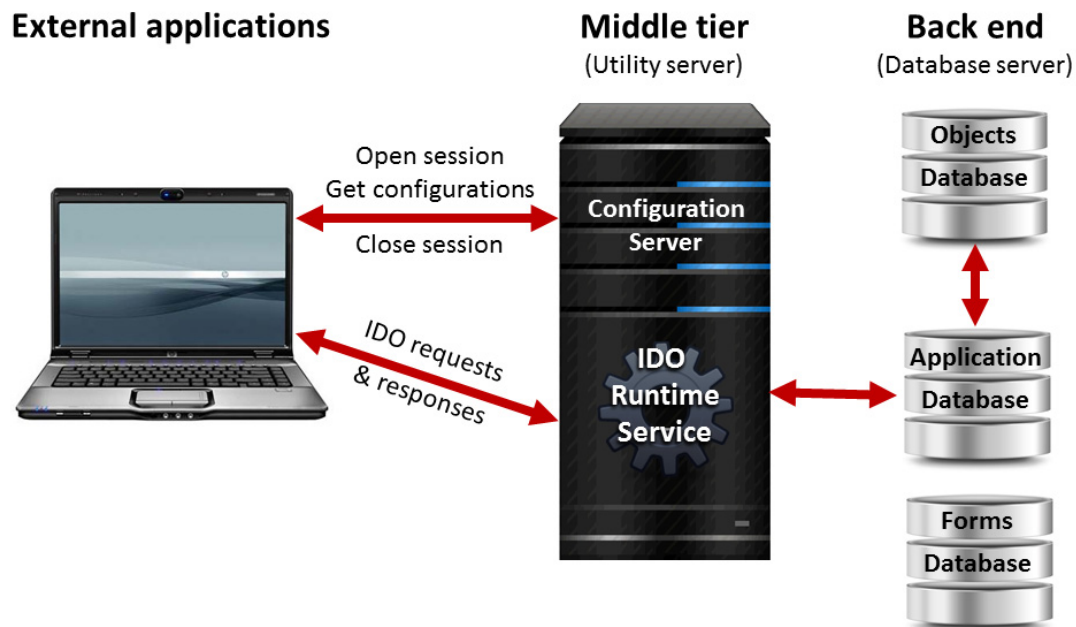
Each time a request for a connection comes in, the Configuration Server returns the URL of the IDO Request Service page on a utility server to the client. The client then sends an "open session" request, and all IDO requests made during that session, to the specified IDO Request Service, which runs the request and sends the IDO response back to the client.

Framework services communication with IDOs



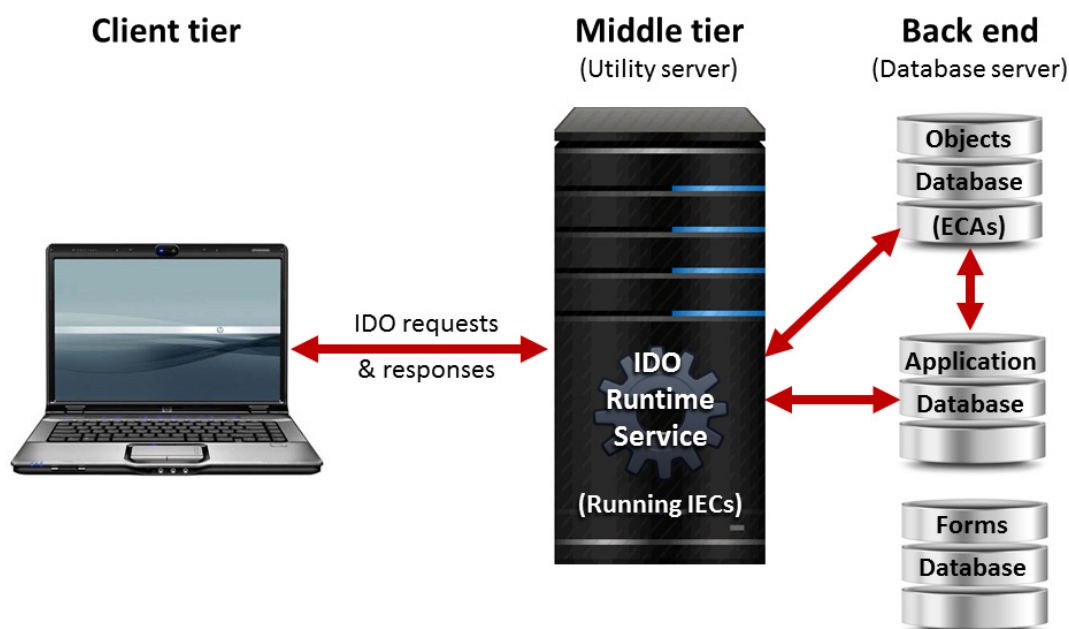
The Mongoose framework services running on the utility server do not require a login, so there is no "open session" request. Instead, the services read an XML file on the utility server that contains the names of the configurations each service monitors. This XML file is updated when an administrator updates the Service Configuration Manager utility.

IDO request/response communication with IDOs



When an external application initiates communication with the IDO runtime, it has to first get the list of configurations. This requires communication with the Configuration Server. The application then opens a session. Once the session has been established, a session ID is passed back to the application. That session ID is used by subsequent IDO requests (GetPropertyInfo, LoadCollection, UpdateCollection, or Invoke), which communicate with the proper configuration's IDO runtime engine. The IDO runtime engine is the "middleman" between the external application and the Mongoose databases. When the communication is finished the external application must then close the session by sending a CloseSession request to the IDO Runtime Service.

IDO extension class communication with IDOs



IDO extension classes are compiled into a .NET class library assembly (ECA) and stored in the Objects Database. The IDO Runtime Service loads these assemblies on demand and calls methods and event handlers in the extension classes in response to IDO requests.

User middleware permissions and licensing

You should create one or more user IDs that can be used for remote sessions that access IDOs through clients other than WinStudio. For instance, when you use IDO request client and IDO protocol libraries to get a connection and subsequent access to an IDO and its methods, the client looks up the permissions on the calling user ID to see if it is permitted to access that IDO, and if so, which types of access are allowed.

Setting permissions for an IDO “user”

To set up the proper permissions for a system “user” that you can use for IDO remote sessions:

- 1 In the **Users** form, create and save the “user” to be used for these transactions.
- 2 Click **User Authorizations**

WinStudio displays the **Object Authorizations for User** form with the IDO “user” selected.

- 3 In the **Object Authorizations for User** form, select **Object Type: MIDDLEWARE**.
- 4 In the **Object Name** field, click the down arrow to display a list of IDOs that are registered for the current site.

This list includes IDOs that are registered as part of the core framework. You can either select a name from the list or type in any IDO name (with prefix).

- 5 Specify the privileges that are granted or revoked for this user ID:
 - **Edit** privilege allows the user to insert, update, or delete records while accessing an IDO.
 - **Execute** privilege allows the user to run IDO methods.
 - **Read** privilege allows the user to view data (for example, run a LoadCollection request).

These middleware permissions have no effect when a user is running WinStudio (the Mongoose smart client), which disables all middleware security and secures everything at the form level. They are in effect only when this user ID is used to access the IDO Runtime Service directly.

Licensing

Customer-created IDOs are assigned to the MGCoreAutomation license module. Imported or extended IDOs are associated with the license modules of the base IDO that they extend.

If you are accessing an IDO either through the XML IDO interface (as with replication, for example) or through the programmatic IDO interface, make sure the user ID specified in the session login data is assigned to a license module that allows access to that IDO.

For more information

For more information about IDOs and related topics, see these additional sources:

For information about...	Look in...
...creating and editing IDOs and their methods and properties	...the online help, IDO-related topics
...the Application Event System	...the online help ...the <i>Guide to the Application Event System</i>
...customizing and modifying forms, including an overview of form development tools, guidelines for customizing and modifying forms, and information on some common development tasks	...the online help ... <i>Infor Mongoose Design Considerations</i>
...the IDO .NET API, the IDO .NET class library, IDO extension classes	...the <i>Infor Mongoose IDO Development Guide</i>
...licensing custom forms and IDOs	...the <i>Multi-Site Planning Guide</i>

For information about...	Look in...
...performing diagnostics on IDO use; viewing logged data	...the IDO Runtime Development Server online help or Log Monitor online help (on the utility server or an administrative client)
...dealing with metadata for IDOs; synchronizing metadata from upgrades or add-on products; importing or exporting IDO-related metadata	...the App Metadata Sync utility online help (on the utility server) ... the App Metadata Transport utility online help (on the utility server)
...defining configurations (named combinations of application, forms, and objects databases)	...the Configuration Manager online help (on the utility server)

Chapter 2: Using XML IDO Request and Response Documents

2

XML request and response documents provide the lowest-level interaction with IDOs. Some applications interfacing with the Mongoose framework expect to receive and submit all data as XML documents. For example, the SL-SunSystems interface uses XML documents to submit requests and responses.

This chapter provides information about how to create and use these XML documents, including:

- What kinds of requests can be used ("About XML IDO requests and responses" on page 25)
- The basic requirements for using these XML documents ("Replicating XML documents" on page 26)
- How to post these XML documents ("Posting XML documents" on page 26)
- How XML request documents are structured ("Request documents" on page 27)
- How XML response documents are structured ("Response documents" on page 27)
- Information required in the header of an XML document ("XML request/response header information" on page 28)
- The commonly used IDO request and response types, with detailed examples ("Request and response types" on page 29)
- Instructions for using dates in XML documents ("Passing dates to the IDO Runtime Service" on page 64)
- How to use dynamic IDOs in XML documents ("Calling dynamic IDOs in XML documents" on page 64)

About XML IDO requests and responses

The IDO Web service accepts XML request documents, processes them, and returns any results in XML response documents.

The following requests are discussed in this chapter:

- **GetConfigurations** – Gets a list of available configurations for a server.
- **OpenSession** – Opens a session in which you can send additional requests.

- **GetPropertyInfo** – Gets a list of properties and their attributes from an IDO. For example, if you ask for the properties of the MGCore.UserNames IDO, the response document returns information like the name of the primary base and any secondary tables used by the collection, and for each property, its data type, label string, class, and so on.
- **LoadCollection** – Retrieves a set of records from a collection.
- **UpdateCollection** – Inserts, updates, or deletes records from a collection.
- **Invoke** – Invokes a method for a collection. The response returns any values and parameters from the method.
- **CloseSession** – Closes the session.

A transaction is created for each UpdateCollection request, and optionally for each Invoke request if the method is identified as requiring a transaction.

Replicating XML documents

If you are sending XML request and response documents to/from other applications by using replication, you must set up *non-transactional* replication, as described in the *Multi-Site Implementation Guide*. For a better understanding of how non-transactional replication works, see the *Replication Reference* guide.

Posting XML documents

When posting XML documents:

1. Provide access to the RequestService ASPX page using a URL specification such as:

http://domain_name/IDORequestService/RequestService.aspx

or

http://ip_address/IDORequestService/RequestService.aspx

This URL specification is entered on the **Intranets** form.

The IDORequestService is set up as a virtual directory during installation of the utility server.

You might also want to make the ASPX file available on the Internet, which requires additional network configuration.

TIP: You can use a GetConfigurations request (see page 29) to find the URL to be used by subsequent request documents.

2. Post the XML documents to the URL, using the HTTP POST protocol.

If you require secure transmissions—for example, when transmitting passwords—use HTTPS.

If you are sending XML documents directly to the IDORequestService (that is, not through the programmatic client class), be aware that the service supports GZip compression. Compression is handled automatically when using the client class.

Request documents

A request document consists of a top level IDORequest element containing one or more RequestHeader elements.

For example:

```
<IDORequest ProtocolVersion="x.yy" [SessionID="sessID"]>
  <RequestHeader Type="RequestType">
    <InitiatorType>InitiatorType</InitiatorType>
    <InitiatorName>InitiatorName</InitiatorName>
    <SourceName>Source</SourceName>
    <TargetName>Target</TargetName>
    <RequestData>
      ...request data...
    </RequestData>
  </RequestHeader>

  <RequestHeader Type="RequestType">...</RequestHeader>

</IDORequest>
```

Replace the information in italics with values as described in “XML request/response header information” on page 28.

Response documents

The response documents have a structure very similar to the request documents. The top level element in the response document is IDOResponse, and it contains one or more ResponseHeader elements corresponding to the RequestHeader elements in the request document.

For example:

```
<IDOResponse ProtocolVersion="x.yy" SessionID="sessID">
  <ResponseHeader Type="RequestType">
    <InitiatorType>InitiatorType</InitiatorType>
    <InitiatorName>InitiatorName</InitiatorName>
    <SourceName>Source</SourceName>
    <TargetName>Target</TargetName>
    <ResponseData>
      ...response data...
    </ResponseData>
  </ResponseHeader>

</IDOResponse>
```

The information in italics is replaced with values as described in the Request/Response Information table on “XML request/response header information” on page 28.

XML request/response header information

Use this table when inserting information into an XML request or response document that accesses the IDOs.

Value	Description
x.yy (Protocol version)	The protocol version supported by the client (on a request document) or the server (on a response document). The absence of this attribute means the client or server is running a version of WinStudio earlier than version 5.00. Version 5.00 was the first version to support this attribute. Some of the functionality described in this chapter does not apply for servers/clients running earlier versions.
<i>SessionID</i>	Unique session ID that is created when an OpenSession request succeeds and is returned in the OpenSession response. All subsequent requests should include the SessionID in the request. For backward compatibility you can omit this attribute and the server will use your Web session to look up the SessionID; however, it might be required in the future, so new code should include this attribute.
<i>RequestType</i>	The type of IDO request. The commonly used requests are: • “GetConfigurations” on page 29 • “OpenSession” on page 34 • “GetPropertyInfo” on page 38 • “LoadCollection” on page 44 • “UpdateCollection” on page 51 • “Invoke” on page 61 • “CloseSession” on page 63 Note: These attribute names are case-sensitive.
<i>InitiatorType</i>	Optional. The type of process that initiated this document. Examples of InitiatorType include: • Form - signals that the request was generated by a form. <i>InitiatorName</i> then must contain the form name. • Replication - signals that the replication process generated the request. <i>InitiatorName</i> is blank, and the <i>SourceName</i> and <i>TargetName</i> contain the source and target sites. For UpdateCollection requests, if InitiatorType is set to Replication, an insert is treated as an update if the record already exists.

Value	Description
<i>InitiatorName</i>	Optional. The name associated with the initiator, if any. This could be a form name or a flow name. For example: <InitiatorName>PurchaseOrders</InitiatorName>
<i>Source</i>	Optional. The name of the site where the request originated.
<i>Target</i>	Optional. The name of the site the request is targeting.
<i>Request Data or Response Data</i>	The contents of this element depends on the data required by the RequestHeader Type attribute. For details, see the following sections describing the specific request types.

Request and response types

The commonly used request and response document are described on these pages:

- “GetConfigurations” on page 29
- “OpenSession” on page 34
- “GetPropertyInfo” on page 38
- “LoadCollection” on page 44
- “UpdateCollection” on page 51
- “Invoke” on page 61
- “CloseSession” on page 63

GetConfigurations

This XML document requests a list of the application configuration names available on the server and the relevant "runtime" configuration information associated with each. Most of the information retrieved in the response is set through the Configuration Manager utility.

This request should be made from a client to IDORequestServer\ConfigServer.aspx. The information that is returned includes the AppURL for the IDORequestService\RequestService.aspx. Use the RequestService.aspx URL for processing the rest of the requests during this session.

For programs integrating via IDORequest XMLs to a single utility server, if you already know the RequestService URL and configuration name, the caller could skip the GetConfigurations call and post requests directly to the RequestServer. However, in order to participate in load balancing, or if connecting through a DMZ server, the client must use the URL returned by the GetConfigurations call, so this is always recommended.

Request data

None

Response data

```
<ResponseData>
  <Config Name="name">
  <AppURL>appurl</AppURL>
    <FormsURL>formsurl</FormsURL>
  <AppName>name</AppName>
    <CompanyName>name</CompanyName>
    <Copyright>copyright</Copyright>
  <Version>version</Version>
  <SplashFilename>bmpfilename</SplashFilename>
  <IconFilename>iconfilename</IconFilename>
  <VBAFilename>bmpfilename</VBAFilename>
  <FormCLSID>clsid</FormCLSID>
  <FormFileExtension>extension</FormFileExtension>
  <LoginOverrideProgID />
    <HelpBasePath>path</HelpBasePath>
  <BaseHelpFile>filename</BaseHelpFile>
  <WallpaperFilename>graphicfilename</WallpaperFilename>
  <WallpaperDisplayOption>option</WallpaperDisplayOption>
    <VertComponentSpacing>charunits</VertComponentSpacing>
  <HorzComponentSpacing>charunits</HorzComponentSpacing>
    <ObjectMenuName>stringname</ObjectMenuName>
    <HelpContentsIndexMenuText>text</HelpContentsIndexMenuText>
    <DeleteExistingPrompt>stringname</DeleteExistingPrompt>
    <DeleteNewPrompt>stringname</DeleteNewPrompt>
    <ProgIDRootAugment />
    <ReportViewerExecutable>exename</ReportViewerExecutable>
    <DefFormBGColor>colorset</DefFormBGColor>
    <DefFormFGColor>colorset</DefFormFGColor>
    <DefNormalBGColor>colorset</DefNormalBGColor>
    <DefNormalFGColor>colorset</DefNormalFGColor>
    <DefReadOnlyBGColor>colorset</DefReadOnlyBGColor>
    <DefReadOnlyFGColor>colorset</DefReadOnlyFGColor>
    <DefRequiredBGColor>colorset</DefRequiredBGColor>
    <DefRequiredFGColor>colorset</DefRequiredFGColor>
    <DefTbdBGColor>colorset</DefTbdBGColor>
    <DefTbdFGColor>colorset</DefTbdFGColor>
    <DefFormFont>fontdef</DefFormFont>
```

```

<UseHtmlHelp>1|0</UseHtmlHelp>
<ActiveScriptingOnly>0|1</ActiveScriptingOnly>
<PrimaryCacheUsesFormCaption>0|1</PrimaryCacheUsesFormCaption>
<ColSecurityPrimaryOnly>0|1</ColSecurityPrimaryOnly>
<RightJustifyNewLabels>0|1</RightJustifyNewLabels>
<ValidationErrorIndicators>0</ValidationErrorIndicators>
<DisableInvalidValuePrompt>0|1</DisableInvalidValuePrompt>
<XPVisualStyle>0</XPVisualStyle>
/Config>
<Config Name="name">
...etc...
</ResponseData>

```

Tag	Description
ConfigGroup	Optional. Parameter in the request header used to specify a configuration group (as defined in the Configuration Manager utility) whose configurations should be passed.
Config	Defines the environment (servers, databases, etc.) that are required for the presentation of an application. Each configuration references an application ID. Name – Configuration name, which is unique.
AppURL	The requestservice URL to use for processing subsequent IDO requests.
FormsURL	URL to the runtime instance of FormServer (providing WinStudio access to a Forms database).
AppName	Displayable name for this application (for example, SyteLine).
CompanyName	Name of the application vendor.
Copyright	Copyright notice for the application.
Version	Version number of the application.
SplashFilename	Filename and path for the application's splash-screen graphic.
IconFilename	Filename and path for the application icon.
VBAFilename	If the application uses Microsoft VB, the filename and path to the icon displayed in the VB editor.
FormCLSID	Not used.
FormFileExtension	Not used.
LoginOverrideProgID	Not used.
HelpBasePath	HTTP path to the base help directory. The application takes this path and adds the Language/culture subfolder (for example, http://installation_folder/Language/en-US) plus the BaseHelpFile name to create the full path.

Tag	Description
BaseHelpFile	Name of the help file opened from the Help menu > Contents and Index option.
WallpaperFilename	Graphic file (of any supported .NET image file type) that displays as background wallpaper in the application window.
WallpaperDisplayOption	Specifies whether the background wallpaper image is centered, tiled, or scaled to fit the application window.
VertComponentSpacing	The amount of vertical spacing between components, in character units. This is used by the application's Arrange Column alignment buttons.
HorzComponentSpacing	The amount of horizontal spacing between components, in character units. This is used by the application's Arrange Column alignment buttons.
ObjectMenuName	Caption of the application's Actions menu. This can be a literal string or the name of a string in the Strings table.
HelpContentsIndexMenuText	Alternative text to use for the Contents and Index command on the application's Help menu.
DeleteExistingPrompt	Specifies text to display when a user tries to delete an existing record. This can be a literal string or the name of a string in the Strings table.
DeleteNewPrompt	Specifies text to display when a user tries to delete an unsaved record. This can be a literal string or the name of a string in the Strings table.
ProgIDRootAugment	Not used.
ReportViewerExecutable	Name of the report viewer executable program to use, if not the default.
Def...Color	Server-side default colors for form components, which can be overridden by the user.
DefFormFont	Server-side default font for forms, which can be overridden by the user.
UseHtmlHelp	Specifies the type of application help: • 0 - Web-based help • 1 - HTML Help (.chm) Note: In current releases, this value should always be 0 .
ActiveScriptingOnly	Not used.
PrimaryCacheUsesFormCaption	Specifies what to use as the application's Actions menu caption if ObjectMenuName is not specified: • 0 - Use the default name, Objects. • 1 - Use the form caption as the menu caption.

Tag	Description
ColSecurityPrimaryOnly	Specify whether insert and delete privileges apply to primary collection and/or subcollections: • 0 - Security permissions apply to primary collection and to secondary or subcollections. • 1 - Security permissions apply to primary collection only.
RightJustifyNewLabels	Specifies how new static components (labels) on forms are to be justified: • 0 - Left justified • 1 - Right justified
ValidationErrorIndicators	Not used.
DisableInvalidValuePrompt	If set to 1 , the user is not prompted to change a value if the validation for a field fails.
XPVisualStyle	Not used.

Example: GetConfigurations

This request finds the available configurations:

Request

```
<IDORequest>
  <RequestHeader Type="GetConfigurations" ConfigGroup="Infor"/>
</IDORequest>
```

Response

```
<IDOResponse ProtocolVersion="5.00" SessionID="">
  <ResponseHeader Type="GetConfigurations">
    <InitiatorType />
    <InitiatorName />
    <SourceName />
    <TargetName />
    <ResponseData>
      <Config Name="MongooseDev">
        <AppURL>http://usserver/IDORequestService/
RequestService.aspx</AppURL>
        <FormsURL />
        <AppName>Application Name</AppName>
        <CompanyName>Name of Company</CompanyName>

        ...additional configuration information...

      </Config>
```

```
    </ResponseData>
  </ResponseHeader>
</IDOResponse>
```

OpenSession

This XML request/response document validates your identity and creates a user session. If the OpenSession is successful, it pulls some information from the configuration file on the server.

You must submit an OpenSession request before you can submit other requests (except for GetConfigurations).

Request data

```
<RequestData>
  <UserId>userid</UserId>
  <LanguageID>culture</LanguageID>
  <ConfigName>config</ConfigName>
  <MachineName>machinename</MachineName>
  <DomainUserName>userID</DomainUserName>
  <ApplicationName>appname</ApplicationName>
  <AllowCloseExistingSessions>>false</AllowCloseExistingSessions>
  <Password Encrypted="N">pswd</Password>
  <Workstation>workstation</Workstation>
</RequestData>
```

Response data

```
<ResponseData>
  <UserId>userid</UserId>
  <LanguageID>culture</LanguageID>
  <ProductVersion>version</ProductVersion>
  <DeadlockRetry>0</DeadlockRetry>
  <License Status="status">
    <Message>message</Message>
  </License>
  <RegionalSettings MessageLanguageID="langID" LocaleID="langID"
    DecimalSeparator="dec-sep" DigitGroupSeparator="dig-sep"
    DigitsInGroup="dig">regional-settings</RegionalSettings>
  <AdditionalFailureInformation>info</AdditionalFailureInformation>
  <ServerDate>datetime</ServerDate>
```

```

    <LoginResult>status</LoginResult>
    <PrimaryGroupName>group</PrimaryGroupName>
    <DaysUntilPasswordExpires>days</DaysUntilPasswordExpires>
    <EditLevel>level</EditLevel>
    <SuperUser>su-indicator</SuperUser>
    <StartupMethods>startup-method-spec</StartupMethods>
    <AuditingEnabled>true|false</AuditingEnabled>
  </ResponseData>

```

Tag	Description
UserId	The user ID, which must be defined for the specified configuration.
LanguageID	Optional. The culture being used in this session. This corresponds to a value in the LanguageIDs table. Default = en-US
ConfigName	The configuration where the session will run. You can get a list of configurations by using the request.
MachineName	Optional. Name of the computer being used in this session. Used primarily for diagnostics.
DomainUserName	Optional. The client's NT user name. Used primarily for diagnostics.
ApplicationName	Optional. The application name being used in this session. Used primarily for diagnostics.
AllowCloseExistingSessions	Optional. If True , existing sessions can be closed if the concurrent login limit prevents user from logging in. Default = False
Password	The user's password. If Encrypted = Y , the password is encrypted; however, Encrypted is usually set to N .
Workstation	Optional. The workstation name, currently only used as an encrypted value by WinStudio.
ProductVersion	The version of the application.
DeadlockRetry	Number of times to retry the connection if a deadlock occurs.
License	If the login is successful, this indicates the status of the application database's license (VALID, INVALID). If INVALID, the message contains additional info, such as when the license expired.
RegionalSettings	Regional settings and locale settings used in this session. This information comes from the LanguageIDs table.
AdditionalFailureInformation	If the login fails, this value may contain additional information about why.

Tag	Description
ServerDate	The server date. WinStudio's CURDATE() and CURTIME() keywords need to yield values that match the server for the application, not the client machine. This ServerDate value is used to compute the difference between the client machine's date and time and the server's, so that each time CURDATE() and CURTIME() is evaluated, WinStudio can produce the correct server-based result.
LoginResult	Result of the specified user's login attempt. Possible values include: • Success • InvalidCredentials • AccountDisabled • AccountLocked • PasswordExpired • PasswordWillExpire • SessionLimit • ConcurrentSessionLimit • InvalidConfiguration • UnknownFailure – Should never happen • LicenseInconsistency – Indicates that someone has tampered with the licensing
PrimaryGroupName	Primary group defined for the specified user on the Users form
DaysUntilPasswordExpires	Days until the specified user's current password expires
EditLevel	Form edit level from the Users form: 0=None 1=Basic 2=Full 3=Site (4=vendor)
SuperUser	Indicates whether the specified user is a SuperUser. 1=Yes 0=No
StartupMethods	Used to set some initial global application variables.
AuditingEnabled	For use by WinStudio.

Example: OpenSession

This example opens a session using the configuration SL_OH for user **jimbow**.

Request

```
<IDORequest>
  <RequestHeader Type="OpenSession">
    <InitiatorType />
    <InitiatorName />
    <SourceName />
    <TargetName />
    <RequestData>
      <UserID>jimbow</UserID>
      <LanguageID>en-US</LanguageID>
      <ConfigName>SL_OH</ConfigName>
      <MachineName>bow_xp</MachineName>
      <DomainUserName>usjimbow</DomainUserName>
      <ApplicationName>WinStudio</ApplicationName>
      <AllowCloseExistingSessions>>false</AllowCloseExistingSessions>
      <Password Encrypted="N" />
      <Workstation />
    </RequestData>
  </RequestHeader>
</IDORequest>
```

Response

```
<IDOResponse ProtocolVersion="5.00" SessionID="0a87afce-3a72-4faa-8b11-f26e72b4057d">
  <ResponseHeader Type="OpenSession">
    <InitiatorType />
    <InitiatorName />
    <SourceName />
    <TargetName />
    <ResponseData>
      <UserID>jcurry</UserID>
      <LanguageID />
      <ProductVersion>8.00.00</ProductVersion>
      <DeadlockRetry>0</DeadlockRetry>
      <License Status="VALID">
        <Message />
      </License>
      <RegionalSettings MessageLanguageID="1033" LocaleID="1033"
        DecimalSeparator="." DigitGroupSeparator="," DigitsInGroup="3">

```

... list of regional settings ...

```
</RegionalSettings>
<AdditionalFailureInformation />
<ServerDate>20070806 15:47:40.983</ServerDate>
<LoginResult>Success</LoginResult>
<PrimaryGroupName />
<DaysUntilPasswordExpires>2147483647</DaysUntilPasswordExpires>
<EditLevel>4</EditLevel>
<SuperUser>1</SuperUser>
<StartupMethods> ... list of startup methods ... </StartupMethods>
<AuditingEnabled>false</AuditingEnabled>
</ResponseData>
</ResponseHeader>
</IDOResponse>
```

GetPropertyInfo

This XML request/response document retrieves property information published by an IDO. For each property in the IDO, the response returns the property's attributes.

Request data

```
<RequestData>
<GetPropertyInfo Name="IDOName" IncludeClassNotesFlag="Y|N" />
</RequestData>
```

Response data

```
<ResponseData>
<GetPropertyInfo Name="IDOName"
IncludeClassNotesFlag="Y|N" InsertPostedToWorkflow="Y|N"
ClassNotesExist="Y|N">
<IDOProperties PrimaryBaseTable="tablename"
PrimaryBaseTableAlias="alias">
<Property Name="prop-name" PropertyClass="prop-class-name" Key="seq"
ProgID="IDOName">
<DataType DecimalPosition="n">type-name</DataType>
<LinkBy><PropertyPair Parent="parentprop" Child="childprop" /></LinkBy>
<Length>nn</Length>
```

```
<ColumnType>sql-column-type</ColumnType>
<ReadOnly />
<Required />
<RORecord />
<Html />
<Subcollection />
<Identity />
<Warning />
<DefaultValue>default-value</DefaultValue>
<Label>label-value</Label>
<LabelStringID />
<DomainIDOName>ido-name</DomainIDOName>
<DomainProperty>prop-name</DomainProperty>
<DomainAdditionalProperties>property-list</
DomainAdditionalProperties>
<BoolTrue>true-value</BoolTrue>
<BoolFalse>>false-value</BoolFalse>
<InputMask>input-mask</InputMask>
<InputMaskPromptChar>input-mask-prompt</InputMaskPromptChar>
<DefaultIMECharset>ime-charset</DefaultIMECharset>
<BinaryDataFormat>binary-format</BinaryDataFormat>
<Case>case-format</Case>
<Justify>justify-format</Justify>
<DateFormat>date-format</DateFormat>
<DisplayDecimalPosition>n</DisplayDecimalPosition>
<NumDigitsInGroup>n</NumDigitsInGroup>
</Property>
</Properties>
</GetPropertyInfo>
</ResponseData>
```

Tag	Description
GetPropertyInfo	● Name - Contains the name of the IDO for which the caller is requesting property information. Multiple GetPropertyInfo tags may be included in the RequestData, for multiple IDOs. ● IncludeClassNotesFlag - Optional. If set to Y in the request, the IDO runtime determines whether there are any class notes associated with the IDO's primary base table and sets the ClassNotesExist attribute in the response to Y or N. Default = N ● InsertPostedToWorkflow - If set to Y in the response, there is a suspending event for inserts associated with this IDO.
IDOProperties	Includes as attributes the primary base table and its alias, as defined in the IDO metadata. This information can be viewed from the IdoCollections form, on the Tables subcollection. It is primarily used internally.
Property	Includes these attributes: ● Name - Property name. ● PropertyClass - Property class associated with this property. ● Key - If the property is a key property, this attribute will be included and is the 1-based key sequence. ● ProgID - Included for sub-collection properties only. This is the IDO name of the subcollection property, for example, UserEmails.EmailTypeDescription. The rest of the elements under this tag correspond to fields on the IDO Properties form.
DataType	Contains one of these values: ● Binary ● Byte ● CONTAINS - Used to specify relationships ● Currency ● Date ● Decimal - Length specified by the Length attribute ● Float ● GUID ● Long Integer ● NumSortedString ● Short Integer ● String - Length specified by the Length attribute

Tag	Description
LinkBy	Valid only in CONTAINS relationship data. This is similar to the WinStudio LINKBY keyword. The information within this pair of tags tells how the CONTAINS relationship data is linked to the parent collection. For example: <pre> <LinkBy> <PropertyPair Child="Item" Parent="Item"/> <!-- Optional: more sets of PropertyPair tags --> </LinkBy> </pre>
ColumnType	Contains the SQL user-defined type of the column to which this property is bound.
ReadOnly	Is included if the property has the read-only attribute set, and omitted otherwise.
Required	Is included if the property has the required attribute set, and omitted otherwise.
RORRecord	Is included if the property has the read-only record flag attribute set, and omitted otherwise.
Html	Is included if the property contains HTML text.
Subcollection	Is included if the property is a subcollection.
Identity	Is included if the property is bound to an identity (auto-incrementing) column, and omitted otherwise.
Warning	Is included if this property contains warning messages for UpdateCollection responses, and omitted otherwise.
DefaultValue	Returns the default value for the specified component.
Label	Returns a string that is the component's label.
LabelStringID	Not used.
DomainIDOName	Contains the domain IDO where the property is found. For example, the DomainIDOName for a unit of measure code is MG.MGUMs.
BoolTrue	Contains a value that represents True in Boolean evaluations of the component.
BoolFalse	Contains a value that represents False in Boolean evaluations of the component.
InputMask	Contains the pattern used for the formatting input mask.
InputMaskPromptChar	Contains the character used in the pattern to signify empty spaces.
DefaultIMECharset	Returns the character-set information for components which receive input from Asian language keyboards.
BinaryDataFormat	Indicates a graphic format: • BMP - bitmap • ICO - icon

Tag	Description
Case	Indicates the case of text: • U - uppercase • L - lowercase • M - mixed case
Justify	Indicates text justification: • L - left • C - center • R - right
DateFormat	Indicates the date format: • D - date • T - time • B - both
DisplayDecimal Position	Indicates the number of digits after the decimal symbol.
NumDigitsInGroup	Indicates the number of digits in a unit group.

Example 1: GetPropertyInfo

This request gets property information for SL.SLItems:

Request

```
<IDORequest>
  <RequestHeader Type="GetPropertyInfo" SessionID="296d2410-4de7-4395-
bd50-58a89df3c3c6">
  <RequestData>
    <GetPropertyInfo Name="MGCore.Users" />
  </RequestData>
</RequestHeader>
</IDORequest>
```

Response

```
<IDOResponse>
  <ResponseHeader Type="GetPropertyInfo" SessionID="296d2410-4de7-4395-
bd50-58a89df3c3c6">
  <ResponseData>
    <GetPropertyInfo>
      <IDOProperties PrimaryBaseTable="item" PrimaryBaseTableAlias="itm">
        <Property Key="1" Name="Item" PropertyClass="ItemBase">
          <DataType Length="30">String</DataType>
          <ColumnType>ItemType</ColumnType>
          <Required/>
          <LabelStringID>sItem</LabelStringID>
        </Property>
        <Property Name="AbcCode" PropertyClass="AbcCode">
          <DataType Length="1">String</DataType>
          <ColumnType>AbcCodeType</ColumnType>
          <LabelStringID>sABCCode</LabelStringID>
        </Property>
        <Property Name="AcceptReq" PropertyClass="ListYesNo">
          <DataType>Byte</DataType>
          <ColumnType>ListYesNoType</ColumnType>
          <LabelStringID>sAcceptRequirements</LabelStringID>
        </Property>
        <Property Name="AltItem" PropertyClass="Item">
          <DataType Length="30">String</DataType>
          <ColumnType>ItemType</ColumnType>
          <LabelStringID>sAlternateItem</LabelStringID>
        </Property>
      </IDOProperties>
      <DomainIDOName>SL.SLItems</DomainIDOName>
      <DomainProperty>Item</DomainProperty>
    </GetPropertyInfo>
  </ResponseData>
</ResponseHeader>
</IDOResponse>
```

```
<DomainAdditionalProperties>Description</DomainAdditionalProperties>
</Property>
...more properties...
</IDOProperties>
</GetPropertyInfo>
</ResponseData>
</ResponseHeader>
</IDOResponse>
```

LoadCollection

This XML request/response document uses the LoadCollection method of an IDO to query for information, and returns the results.

LoadCollection requests and responses can be hierarchical. For example, you can get order header and order lines information in a single request, by nesting the order line LoadCollection requests inside the order header LoadCollection request.

Request data

```
<RequestData>
  <LoadCollection Name="collection" LoadCap="0">
    <LoadType>loadtype</LoadType>
    <ReadMode>mode</ReadMode>
    <Cursor />
    <PropertyList>
      ... list of properties...
    </PropertyList>
    <RecordCap>recordcap</RecordCap>
    <Filter>filter</Filter>
    <OrderBy>order</OrderBy>
    <PostQueryCmd>command</PostQueryCmd>
  </LoadCollection>
</RequestData>
```

Response data

```
<ResponseData>
  <LoadCollection Name="collection" LoadCap="0">
    <ReadMode>mode</ReadMode>
    <Cursor>cursorvalue</Cursor>
```

```
<PropertyList>
  ... list of properties...
</PropertyList>
<LoadType>loadtype</LoadType>
<RecordCap>recordcap</RecordCap>
<Filter>filter</Filter>
<OrderBy>order</OrderBy>
<PostQueryCmd>command</PostQueryCmd>
<Users>
  <UserID="userID">
    <P>propertyvalue</P>
    <P>propertyvalue</P>
    <P>...etc...</P>
  </UserID>
  <Username="User name">
    <P>propertyvalue</P>
    <P>propertyvalue</P>
    <P>...etc...</P>
  </Username>
</Users>
</LoadCollection>
</ResponseData>
```

Tag	Description
LoadCollection	• Name - The collection name (for example MGCore.GroupNames). • LoadCap - Applies only to subcollections. This is the maximum number of subcollections to include in the result. Values for LoadCap are: • -1 - include all subcollections • 1 (the default) - only query the subcollection records for the first parent item • Any other number - Specify the maximum number of subcollections to retrieve.
ReadMode	Specifies the collection read mode, which controls the isolation level used when executing queries (see the online help for Process Defaults). Options are: • ReadCommitted • ReadUncommitted • Default If the tag is empty or omitted, Default is assumed. The read mode appears in the response if the ReadMode tag in the request is anything but Default. This tag does not apply to custom load methods.
LoadType	Specify one of these types: • FIRST • NEXT • PREV • LAST The first time you submit a LoadCollection request, it must be called FIRST or LAST. For subsequent requests, you can use NEXT or PREV.
Cursor	The response data can include a cursor value that is a "bookmark" into the collection to tell where the retrieval stopped. If you will be making multiple requests using NEXT or PREV to retrieve data, include a single <cursor /> tag in your FIRST request. The response document provides a cursor value. Include the response document's cursor value (as <cursor>value</cursor>) in the NEXT request document, so the system can tell where to start the next retrieval request. Continue updating the cursor value as you provide additional NEXT or PREV requests. A LAST request should use the empty tag <cursor />.

Tag	Description
PropertyList	In request data, you only need to list the properties you want returned. The response data mimics the request list. For example: <pre><PropertyList> <Property>name1</Property> ... </PropertyList></pre>
RecordCap	Determines how many records are retrieved in one request. Values are: • -1 (Default) - 200 records are retrieved • 0 - Uncapped - all records are retrieved • Any other number - The specified number of records are retrieved.
Item	The ID attribute contains a value used within the system to identify the item.
propertyvalue	For each item, this list contains values corresponding the properties listed above.
LinkBy	Valid only in CONTAINS relationships. This is similar to the WinStudio LINKBY keyword. The information within this pair of tags tells how the CONTAINS relationship is linked to the parent collection. For example: <pre><LinkBy> <PropertyPair Child="Item" Parent="Item"/> <!-- Optional: More sets of PropertyPair tags --> </LinkBy></pre>
Distinct	Retrieves a set of data representing only the distinct combinations of properties requested. For example: <pre><LoadCollection Name="MGCore.Users"> <LoadType>FIRST</LoadType> <PropertyList> <UserID/> </PropertyList> <Distinct/> </LoadCollection></pre>
Filter	Restricts the result set, similar to the WinStudio FILTERPERM keyword. Any valid SQL where clause syntax is allowed. Some examples: <pre><Filter>CoNum like 'ABC%'</Filter> <Filter>QtyOnHand < 20 AND Whse = 'OH'</Filter></pre> In the response document, this element contains any IDO-level filter specifications that were used.

Tag	Description
OrderBy	Provides a comma-delimited list of properties that specify how the response document result set should be sorted. The DESC keyword may appear after a property name to sort that property descending. For example: <pre> <LoadCollection Name="MGCore.Users"> <LoadType>FIRST</LoadType> <PropertyList> <UserID /> <Username /> <Location /> </PropertyList> <OrderBy>UserID, Username</OrderBy> </LoadCollection> </pre>
PostQueryCmd	Specifies a method to execute once for each row in the result set after the query is completed. This is the equivalent of the WinStudio PQ option in LoadSave overrides and uses the same syntax. For example: <pre> <PostQueryCmd>MyPostQuerySp(Property1, Property2, REF Property3)</PostQueryCmd> </pre>

Example 1: LoadCollection

This request retrieves the task type, task code, and description from the first 200 records in the MGCore.TaskTypes collection.

Request

```

<IDORequest>
  <RequestHeader Type="LoadCollection" SessionID="296d2410-4de7-4395-
bd50-58a89df3c3c6">
    <RequestData>
      <LoadCollection Name="MGCore.TaskTypes">
        <LoadType>FIRST</LoadType>
        <PropertyList>
          <TaskType />
          <TaskCode />
          <TaskDesc />
          <NoteExistsFlag />
        </PropertyList>
        <RecordCap>-1</RecordCap>
      </LoadCollection>
    </RequestData>
  </RequestHeader>

```



```
</IDORequest>
```

Response

```
<IDOResponse>
  <ResponseHeader Type="LoadCollection" SessionID="296d2410-4de7-
4395-bd50-58a89df3c3c6">
    <ResponseData>
      <LoadCollection Name="MGCore.TaskTypes">
        <PropertyList>
          <TaskType />
          <TaskCode />
          <TaskDesc />
          <NoteExistsFlag />
        </PropertyList>
        <Items>
          <Item ID="TaskTypes @0.D='2007-04-18 16:31:53.730'
@0.P='C444D3B2-FB2E-4416-97E2-D5BCA7F7FA74'">
            <P>11001</P>
            <P>JC2</P>
            <P>WebSvc test2</P>
            <P>0</P>
          </Item>
        </Items>
      </LoadCollection>
    </ResponseData>
  </ResponseHeader>
</IDOResponse>
```

Example 2: Custom LoadCollection request

This is a variation of LoadCollection, allowing you to create a custom LoadCollection. It includes this tag:

```
<CustomLoadMethod Name="methodname">
```

and then a parameters list.

Request

```
<IDORequest>
  <RequestHeader Type="LoadCollection" SessionID="296d2410-4de7-4395-
bd50">
    <RequestData>
      <LoadCollection Name="MGCore.UserNames">
        <CustomLoadMethod Name="UserLov">
```

```
<Parameters>
  <Parameter>xxzz%</Parameter>
</Parameters>
</CustomLoadMethod>
<LoadType>FIRST</LoadType>
<PropertyList>
  <Username />
</PropertyList>
<RecordCap>-1</RecordCap>
</LoadCollection>
</RequestData>
</RequestHeader>
</IDORequest>
```

Example 3: Nested LoadCollection request

This request includes CONTAINS relationship records for all parent items.

Request

```
<RequestData>
  <LoadCollection Name="SL.SLItemcusts" SessionID="296d2410-4de7-4395-bd50">
    <LoadType>FIRST</LoadType>
    <PropertyList>
      <CustNum />
      <CustItem />
      <Item />
    </PropertyList>
    <Filter />
    <RecordCap>0</RecordCap>
    <LoadCollection Name="SL.SLItems" LoadCap="-1" />
    <LinkBy>
      <PropertyPair Child="Item" Parent="Item" />
    </LinkBy>
    <LoadType>FIRST</LoadType>
    <PropertyList>
      <Item />
      <ProductCode />
      <DerQtyOnHand />
    </PropertyList>
    <Filter />
    <RecordCap>0</RecordCap>
  </LoadCollection>
```

```
</LoadCollection>
</RequestData>
```

UpdateCollection

This XML request/response document modifies a collection (inserting, updating, or deleting records) using the UpdateCollection method of an IDO.

When performing an update or deletion request, identify the records you want to update or delete. There are two ways to do this:

- **By item ID** - To update or delete by item ID, you must perform a LoadCollection first to retrieve the item ID, and then include it in the `<Item>` node's ID attribute when performing the UpdateCollection update or delete.
- **By the key data** - In this case, specify the attribute `UseKeys="Y"` on the `<Item>` node and set the appropriate properties to the key data when performing the update or delete.

For insert requests, this information is not needed.

UpdateCollection requests and responses can be hierarchical. For example, you can update the order header and order lines in a single request, by nesting the lines' UpdateCollection requests inside the header UpdateCollection request.

Request data

```
<RequestData>
  <UpdateCollection Name="Collection" RefreshAfterUpdate="Y"
    [TxnScope="Item"      | "Collection"]>
    <Items>
      <Item Action="ActionType" ItemNo="recordnumber" [ID=itemID |
        UseKeys="Y"]>
        <Property Name="property" Modified="Y">data</Property>
        <Property Name="property" Modified="Y">data</Property>
        more properties...
      </Item>
      more items...
    </Items>
  </UpdateCollection>
</RequestData>
```

Response data

```
<ResponseData>
  <UpdateCollection Name="Collection" RefreshAfterUpdate="Y">
```

```

<Items>
  <Item Action="ActionType" ItemNo="recordnumber" ID="itemID
    Id.RecordDate='date'
    Id.RowPointer='row-pointer'">
    <Property Name="property" Modified="Y">data</Property>
    <Property Name="property" Modified="Y">data</Property>
    more properties...
  </Item>
  more items...
</Items>
</UpdateCollection>
</ResponseData>

```

Tag	Description
UpdateCollection	The tag can contain these attributes: • Name - Contains the name of the IDO collection. • RefreshAfterUpdate - Y indicates that the response document should show information for inserted or updated items that was refreshed after the update was done. (Deleted items are not refreshed.) N indicates that the response document information is not refreshed after the update. • TxnScope - Optional. Can be set to Collection (the default value) or Item. When this is set to Item, each individual item in the UpdateCollection request is saved in a separate transaction. For a hierarchical (nested) UpdateCollection request, the value of the TxnScope attribute at the root level determines the behavior for the entire UpdateCollection request. If an exception occurs while processing an UpdateCollection request using the Item TxnScope, records that were saved before the exception are still committed, but no additional records are saved. If TxnScope is omitted or is set to Collection, the request is processed in a single transaction, as in previous versions. This attribute has no effect when it is included in an inner UpdateCollection within hierarchical requests.
CollectionID	Optional. Used when reporting errors that occur while processing items in an UpdateCollection request. If CollectionID is omitted in the request, IDOName is used instead in any error response. An UpdateCollection request can contain hierarchical (nested) requests. Having a collection ID allows the system response to include both the error message and which item caused the error. (See example 3 below.)

Tag	Description
Item	The tag can contain these attributes: • Action - The action type can be Insert, Update or Delete. For Inserts, if the InitiatorType (see page 28) is set to "Replication" the insert is treated as an update if the record already exists. • ItemNo - This number is user-defined and optional. It must be a valid 32-bit signed integer. If RefreshAfterUpdate is Y and this number is specified, the response includes the record number value, so you can match updated records with the originals and see more easily what has changed. • ID - This value is used within the system to identify the item. When requesting an update or deletion of existing records, include either the item ID or the UseKeys attribute (see below). Omitting the ID attribute defaults to the same behavior as UseKeys="Y". • UseKeys - Y (the default value) indicates that the update or delete request will use key data rather than item IDs. N indicates that the request will include item IDs.
Property	Include in your request any properties that you plan to update, as well as all non-nullable properties that do not have a default value. You can omit properties that have a default value and that will not change as a result of the update. The tag can contain these attributes: • Name - This contains the name of the property. • Modified - This value specifies whether the property was updated. Only values flagged as "Modified=Y" will be updated in the database.
LinkBy	Valid only in CONTAINS relationships. Similar to the WinStudio LINKBY keyword. The information within this pair of tags tells how the CONTAINS relationship is linked to the parent collection. For example: <pre><LinkBy> <PropertyPair Child="Item" Parent="Item"/> <!-- Optional: more sets of PropertyPair tags --> </LinkBy></pre>

TIP: You might want to include in your request some items or properties that you are not directly updating. For example, a total related to a quantity that was modified may change as a result of the quantity changing. Your request should therefore include the total, as well as the quantity.

Example 1: UpdateCollection – Insert, Update and Delete Items

Request

```
<IDOResponse>
```

```
<RequestHeader Type="UpdateCollection" SessionID="296d2410-4de7-
4395-bd50-58a89df3c3c6">
  <RequestData>
    <UpdateCollection Name="MGCore.TaskTypes"
RefreshAfterUpdate="Y">
      <Items>
        <Item Action="Insert" ItemNo="0">
          <Property Name="TaskType" Modified="Y">11001</Property>
          <Property Name="TaskCode" Modified="Y">JC2</Property>
          <Property Name="TaskDesc" Modified="Y">WebSvc test2</
Property>
        </Item>
        <Item Action="Update" ItemNo="1">
          ... properties...
        </Item>
        <Item Action="Delete" ItemNo="2">
          ... properties...
        </Item>
      </Items>
    </UpdateCollection>
  </RequestData>
</RequestHeader>
</IDORequest>
```

Response

```
<IDOResponse>
  <ResponseHeader Type="UpdateCollection" SessionID="296d2410-4de7-
4395-bd50-58a89df3c3c6">
    <ResponseData>
      <UpdateCollection Name="MGCore.TaskTypes"
RefreshAfterUpdate="Y">
        <Items>
          <Item Action="Insert" ItemNo="0" ID="PBT=TaskTypes
TaskTypes.RecordDate='2007-04-18 16:31:53.730'
TaskTypes.RowPointer='C444D3B2-FB2E-4416-97E2-D5BCA7F7FA74'">
            <Property Name="TaskType" Modified="Y">11001</Property>
            <Property Name="TaskCode" Modified="Y">JC2</Property>
            <Property Name="TaskDesc" Modified="Y">WebSvc test2</
Property>
          </Item>
          <Item Action="Update" ItemNo="1">
            ... properties...
          </Item>
          <Item Action="Delete" ItemNo="2">
```

```
        ... properties...
    </Item>
</Items>
</UpdateCollection>
</ResponseData>
</ResponseHeader>
</IDOResponse>
```

Example 2: UpdateCollection - Nested insert of group and users

This request inserts a user-defined type and some values for it.

Request

```
<IDORequest ProtocolVersion="5.00" SessionID="296d2410-4de7-4395-
bd50-58a89df3c3c6">
  <RequestHeader Type="UpdateCollection">
    <InitiatorType>Form</InitiatorType>
    <InitiatorName>UserDefinedTypes</InitiatorName>
    <SourceName />
    <TargetName />
    <RequestData>
      <UpdateCollection Name="MGCore.UserDefinedTypes"
RefreshAfterUpdate="Y">
        <CollectionID />
        <Items>
          <Item ID="" ItemNo="1" Action="Insert">
            <Property Name="Name">ColorType</Property>
            <Property Name="NoteExistsFlag" Modified="N" />
            <Property Name="RowPointer" Modified="N" />
            <Property Name="_ItemWarnings" Modified="N" />
            <Property Name="Description">New type</Property>
            <UpdateCollection Name="MGCore.UserDefinedTypeValues"
RefreshAfterUpdate="Y">
              <LinkBy>
                <PropertyPair Parent="Name" Child="TypeName" />
              </LinkBy>
              <CollectionID>UserDefinedTypeValues</CollectionID>
              <Items>
                <Item ID="" ItemNo="0" Action="Insert">
                  <Property Name="Value">Red</Property>
                  <Property Name="TypeName">ColorType</Property>
                  <Property Name="NoteExistsFlag" Modified="N" />
                  <Property Name="RowPointer" Modified="N" />
                  <Property Name="_ItemWarnings" Modified="N" />
```

```
        <Property Name="Description">red</Property>
    </Item>
    <Item ID="" ItemNo="1" Action="Insert">
        <Property Name="Value">Green</Property>
        <Property Name="TypeName">ColorType</Property>
        <Property Name="NoteExistsFlag" Modified="N" />
        <Property Name="RowPointer" Modified="N" />
        <Property Name="_ItemWarnings" Modified="N" />
        <Property Name="Description">green</Property>
    </Item>
</Items>
</UpdateCollection>
</Item>
</Items>
</UpdateCollection>
</RequestData>
</RequestHeader>
</IDORequest>
```

Response

```
<IDOResponse ProtocolVersion="5.00" SessionID="296d2410-4de7-4395-
bd50-58a89df3c3c6">
    <ResponseHeader Type="UpdateCollection">
        <InitiatorType />
        <InitiatorName />
        <SourceName />
        <TargetName />
        <ResponseData>
            <UpdateCollection Name="MGCore.UserDefinedTypes"
RefreshAfterUpdate="Y">
                <CollectionID />
                <Items>
                    <Item ID="PBT=[UserDefinedTypes] UserDefinedTypes.DT=[20070806
15:18:06.653] UserDefinedTypes.ID=[a955821a-67d2-4771-bf99-
89ff949e9f6e]" ItemNo="1" Action="Insert">
                        <Property Name="Name" Modified="N">ColorType</Property>
                        <Property Name="NoteExistsFlag" Modified="N">0</Property>
                        <Property Name="RowPointer" Modified="N">a955821a-67d2-4771-
bf99-89ff949e9f6e</Property>
                        <Property Name="_ItemWarnings" Modified="N" />
                        <Property Name="Description" Modified="N">my new type</Property>
                    <UpdateCollection Name="MGCore.UserDefinedTypeValues"
RefreshAfterUpdate="Y">
```



```

<LinkBy>
  <PropertyPair Parent="Name" Child="TypeName" />
</LinkBy>
<CollectionID />
<Items>
  <Item ID="PBT=[UserDefinedTypeValues]
UserDefinedTypeValues.DT=[20070806 15:18:06.683]
UserDefinedTypeValues.ID=[6bbfb52d-20aa-40d6-a1d6-f5596f934c30]"
ItemNo="0" Action="Insert">
    <Property Name="Value" Modified="N">Red</Property>
    <Property Name="TypeName" Modified="N">ColorType</Property>
    <Property Name="NoteExistsFlag" Modified="N">0</Property>
    <Property Name="RowPointer" Modified="N">6bbfb52d-20aa-40d6-
a1d6-f5596f934c30</Property>
    <Property Name="_ItemWarnings" Modified="N" />
    <Property Name="Description" Modified="N">red</Property>
  </Item>
  <Item ID="PBT=[UserDefinedTypeValues]
UserDefinedTypeValues.DT=[20070806 15:18:06.810]
UserDefinedTypeValues.ID=[01ef0dd7-2c28-43b7-8c72-0d9afe742ef4]"
ItemNo="1" Action="Insert">
    <Property Name="Value" Modified="N">Green</Property>
    <Property Name="TypeName" Modified="N">ColorType</Property>
    <Property Name="NoteExistsFlag" Modified="N">0</Property>
    <Property Name="RowPointer" Modified="N">01ef0dd7-2c28-43b7-
8c72-0d9afe742ef4</Property>
    <Property Name="_ItemWarnings" Modified="N" />
    <Property Name="Description" Modified="N">green</Property>
  </Item>
</Items>
</UpdateCollection>
</Item>
</Items>
</UpdateCollection>
</ResponseData>
</ResponseHeader>
</IDOResponse>

```

Example 3: UpdateCollection error response

If you try to insert an item that already exists in the collection, the response returns an error indicating the item where the problem occurred. The ErrorMessage tag has an attribute named **ItemNo**, in this format:

```
<top-level-ItemNo>[<collection-id>.<child-collection-ItemNo>[...]]
```

If CollectionID is omitted in the request, IDOName is used instead.

Request

```
<IDORequest ProtocolVersion="5.00" SessionID="11d39624-5d71-436f-8554-708922b0alf4">
  <RequestHeader Type="UpdateCollection">
    ...header information...
    <RequestData>
      <UpdateCollection Name="MGCore.UserDefinedTypes"
RefreshAfterUpdate="Y">
        <CollectionID />
        <Items>
          <Item ID="PBT=[UserDefinedTypes] UserDefinedTypes.DT=[20070806
15:18:06.653] UserDefinedTypes.ID=[a955821a-67d2-4771-bf99-89ff949e9f6e]" ItemNo="23" Action="Update">
            <Property Name="Name" Modified="N">ColorType</Property>
            <Property Name="NoteExistsFlag" Modified="N">0</Property>
            <Property Name="RowPointer" Modified="N">a955821a-67d2-4771-
bf99-89ff949e9f6e</Property>
            <Property Name="_ItemWarnings" Modified="N" />
            <Property Name="Description" Modified="N">New type</Property>
            <UpdateCollection Name="MGCore.UserDefinedTypeValues"
RefreshAfterUpdate="Y">
              <LinkBy>
                <PropertyPair Parent="Name" Child="TypeName" />
              </LinkBy>
              <CollectionID>UserDefinedTypeValues</CollectionID>
              <Items>
                <Item ID="" ItemNo="2" Action="Insert">
                  <Property Name="Value">Red</Property>
                  <Property Name="TypeName">ColorType</Property>
                  <Property Name="NoteExistsFlag" Modified="N" />
                  <Property Name="RowPointer" Modified="N" />
                  <Property Name="_ItemWarnings" Modified="N" />
                  <Property Name="Description">duplicate</Property>
                </Item>
              </Items>
            </UpdateCollection>
          </Item>
        </Items>
      </UpdateCollection>
    </RequestData>
  </RequestHeader>
</IDORequest>
```

Response

```
<IDOResponse ProtocolVersion="5.00" SessionID="11d39624-5d71-436f-8554-708922b0a1f4">
  <ResponseHeader Type="UpdateCollection">
...header information
  <ResponseData />
  <ErrorMessage ItemNo="23.UserDefinedTypeValues.2">The Type Name,
Value combination entered already exists.</ErrorMessage>
  </ResponseHeader>
</IDOResponse>
```

Example 4: UpdateCollection – Update using item IDs

This request updates existing records using the **ID** attribute. To find a specific ID, perform a LoadCollection request on the data before performing the UpdateCollection.

Request

```
<IDORequest>
  <RequestHeader Type="UpdateCollection" SessionID="296d2410-4de7-4395-bd50-58a89df3c3c6">
  <RequestData>
    <UpdateCollection Name="MGCore.TaskTypes" RefreshAfterUpdate="Y">
      <Items>
        <Item Action="Update" ItemNo="0" ID="TaskTypes @0.D='2004-03-08
15:18:35.897' @0.P='32537FE4-64D4-4BD5-8C9B-8A5EA27D6A33'">
          <Property Name="TaskType" Modified="N">11001</Property>
          <Property Name="TaskCode" Modified="N">T001</Property>
          <Property Name="TaskDesc" Modified="Y">Updated TaskCode</
Property>
        </Item>
      </Items>
    </UpdateCollection>
  </RequestData>
</RequestHeader>
</IDORequest>
```

Response

```
<IDOResponse>
  <ResponseHeader Type="UpdateCollection" SessionID="296d2410-4de7-4395-bd50-58a89df3c3c6">
  <ResponseData>
```

```
<UpdateCollection Name="MGCore.TaskTypes" RefreshAfterUpdate="Y">
  <Items>
    <Item Action="Update" ItemNo="0" ID="PBT=TaskTypes
TaskTypes.RecordDate='2004-05-11 15:25:31.753'
TaskTypes.RowPointer='32537FE4-64D4-4BD5-8C9B-8A5EA27D6A33'">
      <Property Name="TaskDesc">Updated TaskCode</Property>
    </Item>
  </Items>
</UpdateCollection>
</ResponseData>
</ResponseHeader>
</IDOResponse>
```

Example 5: UpdateCollection – Update using keys

This request updates existing records using the **UseKeys** attribute:

Request

```
<IDOResponse>
  <RequestHeader Type="UpdateCollection" SessionID="296d2410-4de7-
4395-bd50-58a89df3c3c6">
    <RequestData>
      <UpdateCollection Name="MGCore.TaskTypes" RefreshAfterUpdate="Y">
        <Items>
          <Item Action="Update" ItemNo="0" UseKeys="Y">
            <Property Name="TaskType" Modified="N">11001</Property>
            <Property Name="TaskCode" Modified="N">T001</Property>
            <Property Name="TaskDesc" Modified="Y">Updated TaskCode</
Property>
          </Item>
        </Items>
      </UpdateCollection>
    </RequestData>
  </RequestHeader>
</IDOResponse>
```

Response

```
<IDOResponse>
  <ResponseHeader Type="UpdateCollection" SessionID="296d2410-4de7-
4395-bd50-58a89df3c3c6">
    <ResponseData>
      <UpdateCollection Name="MGCore.TaskTypes" RefreshAfterUpdate="Y">
        <Items>
```

```
<Item Action="Update" ItemNo="0" ID="PBT=TaskTypes
TaskTypes.RecordDate='2004-05-11 15:25:31.753'
TaskTypes.RowPointer='32537FE4-64D4-4BD5-8C9B-8A5EA27D6A33'">
  <Property Name="TaskDesc">Updated TaskCode</Property>
</Item>
</Items>
</UpdateCollection>
</ResponseData>
</ResponseHeader>
</IDOResponse>
```

Invoke

This XML request/response document invokes a method published by an IDO.

Request data

```
<RequestData>
  <Name>IDOName</Name>
  <Method>MethodName</Method>
  <Parameters>
    <Parameter>PRIMARY</Parameter>
    <Parameter ByRef="Y"/>
    <Parameter ByRef="Y"/>
    <Parameter ByRef="Y"/>
  </Parameters>
</RequestData>
```

Response data

```
<ResponseData>
  <Name>IDOName</Name>
  <Method>MethodName</Method>
  <Parameters>
    <Parameter>PRIMARY</Parameter>
    <Parameter ByRef="Y">n</Parameter>
    <Parameter ByRef="Y">parm</Parameter>
    <Parameter ByRef="Y"/>
  </Parameters>
  <ReturnValue>n</ReturnValue>
</ResponseData>
```

Tag	Description
Name	The name of the IDO that contains the method
Method	The name of the method to invoke
ReturnValue	In the response, contains the return value from the method
Parameter	The name of a parameter for the method. The tag can contain these attributes: • VT - Optional. The variant data type of the parameter (see Microsoft documentation for more information). • ByRef - Optional. Parameters that are marked as ByRef in the IDO metadata will be returned in the response, regardless of the ByRef value in the request. If set to Y, the parameter is input/output. If set to N, or if the attribute is omitted, the parameter is input-only. Note: Parameters should be listed in the order that the method is expecting them. If you use the obsolete Seq attribute, it is ignored.

Example: Invoke

This request determines whether the Group ID "Developers" is valid.

Request

```
<IDORequest ProtocolVersion="5.00" SessionID="296d2410-4de7-4395-  
bd50-58a89df3c3c6">  
  <RequestHeader Type="Invoke">  
    <RequestData>  
      <Name>MGCore.GroupNames</Name>  
      <Method>GroupValid</Method>  
      <Parameters>  
        <Parameter>Developers</Parameter>  
        <Parameter ByRef="Y"/>  
        <Parameter ByRef="Y"/>  
        <Parameter ByRef="Y"/>  
      </Parameters>  
    </RequestData>  
  </RequestHeader>  
</IDORequest>
```

Response

```
<IDOResponse ProtocolVersion="5.00" SessionID="296d2410-4de7-4395-  
bd50-58a89df3c3c6">>
```

```
<ResponseHeader Type="Invoke">
  <ResponseData>
    <Parameters>
      <Parameter VT="8" Seq="0">Developers</Parameter>
      <Parameter VT="1" ByRef="Y" Seq="1"/>
      <Parameter VT="1" ByRef="Y" Seq="2"/>
      <Parameter VT="8" ByRef="Y" Seq="3">The Group Name entered is not
valid.</Parameter>
    </Parameters>
    <ReturnValue VT="2">16</ReturnValue>
  </ResponseData>
</ResponseHeader>
</IDOResponse>
```

CloseSession

This XML request/response document closes the current session.

Request Data

none

Response Data

none

Example: CloseSession

Request

```
<IDOResponse ProtocolVersion="5.00" SessionID="296d2410-4de7-4395-
bd50-58a89df3c3c6">>
  <RequestHeader Type="CloseSession">
    <InitiatorType />
    <InitiatorName />
    <SourceName />
    <TargetName />
    <RequestData />
  </RequestHeader>
</IDOResponse>
```

Response

```
<IDOResponse ProtocolVersion="5.00" SessionID="296d2410-4de7-4395-  
bd50-58a89df3c3c6">  
  <ResponseHeader Type="CloseSession">  
    <InitiatorType />  
    <InitiatorName />  
    <SourceName />  
    <TargetName />  
    <ResponseData />  
  </ResponseHeader>  
</IDOResponse>
```

Passing dates to the IDO Runtime Service

When passing dates to the IDO Runtime Service (through XML documents, .NET, or programmatically through another application), always use this locale-neutral format:

YYYYMMDD HH:MM:SS (.mmm is optional)

Other formats might work for all or some regional settings, depending on how they are used; but this format always works.

Calling dynamic IDOs in XML documents

XML documents can directly query or update the SQL database tables by calling dynamic IDOs. Through dynamic IDOs, you can also directly invoke stored procedures from the SQL database. To access a dynamic IDO, use the standard XML syntax, except:

- For collection requests, instead of the collection name, you specify the table name (in the format `TABLE!table_name`). For example, the XML UpdateCollection request could contain a tag like this:

```
<UpdateCollection Name="TABLE!item">
```

The properties specified in the calls are actually the column names in the table.

- When invoking a method, instead of the IDO name, specify **SP!** and instead of the method name, specify the stored procedure name. For example, the XML Invoke request would contain these tags:

```
<Name>SP!</Name>
```

```
<Method>CalcHrsSp</Method>
```


- When you open the session where you make these requests, you specify a configuration. The configuration points to an application database. This is the database where the requests look for the tables and stored procedures.

Note: Dynamic IDOs are not valid for tables in an outrigger database.

You can also invoke stored procedures by using dynamic IDOs in custom events defined for Mongoose forms. For more information about this, see *Infor Mongoose Design Considerations*.

Infor Mongoose is enabled, “out of the box”, to integrate with Infor ION Connect (commonly known as “ION”). ION is an enterprise messaging system for integrating Infor applications with other Infor and non-Infor applications. Many of these ION-based interactions deal with exchanges of data stored in IDO collections.

Communications between Mongoose-based applications and other applications can be sent as business object documents (BODs) that are routed through ION. These BODs are essentially specialized XML request and response documents, similar to those discussed in Chapter 2, “Using XML IDO Request and Response Documents”.

This chapter provides information about integrating Mongoose-based applications with ION, including:

- “Overview: Mongoose, ION, and BODs” on page 64
- “Setting up Mongoose for ION” on page 65
- “Setting up ION for Mongoose” on page 69

Overview: Mongoose, ION, and BODs

Mongoose-ION integrations work on many of the same principles and practices as other XML request-response documents, such as the multi-site capabilities of Mongoose. They follow a similar structure. They use the same replication system in Mongoose to process XML request and response documents.

The differences lie primarily in:

- The use of Business Object Documents (BODs), as opposed to other types of XML request/response documents
- The way those BODs are processed, as opposed to other types of XML request/response documents
- How BODs are passed to/from the other application, using ION instead of direct interactions
- The fact that BODs are based on OAGIS (Open Applications Group Integration Specification) standards:
 - **BOD nouns** identify the business-specific data that is being exchanged. This data is sometimes called the “business object.” OAGIS has defined an extensive list of standard nouns. Examples include PurchaseOrder, SalesOrder, Quote, Route, and Shipment.
 - **BOD verbs** identify the specific action that is to be performed with the noun data. Again, OAGIS has defined a set of standard verbs, including Sync, Process, Cancel, and Add.

For more information about BOD standards, see the OAGIS Web site (www.oagi.org).

ION integration can include one or more Mongoose application instances linked to one or more BOD-enabled applications through ION Connect. For Mongoose applications, one instance is established for each site.

The mapping between Mongoose application sites and other applications are accomplished using ION “connection points.” This mapping should be a reflection of the business relationship between the databases.

Mongoose-ION exchanges

Mongoose-ION exchanges can be either “outbound” or “inbound.” The exact nature of those exchanges depend on both the BOD noun and BOD verb specifications. We describe here only a general overview of the processes involved.

Outbound operations

An outbound operation typically begins when a user performs an action that triggers a response that requires a data exchange with an external application. If this exchange has been configured to use ION and the built-in BOD system, then Mongoose uses a pre-defined replication document to transform the data from the IDO to XML and build the required BOD containing the data.

These BODs are placed in a table known as the Replication Document Outbox for pickup by ION. At scheduled intervals then, ION retrieves the BODs from the Outbox and moves them to the destination application's BOD "inbox."

Inbound operations

Inbound operations are basically the reverse: ION picks up BODs from a sending application's BOD "outbox" and delivers them to Mongoose's Replication Document Inbox table for processing.

At scheduled polling intervals, Mongoose then retrieves the BOD from the Inbox and uses a pre-defined replication document to map the data from the inbound BOD to the appropriate locations in a specified IDO table.

Information needed for the integration

When planning for the integration, you will need to gather this information:

- The name of the Mongoose application instance (logical ID) that is to be used when you connect to ION
- The tenant ID to be used to connect both applications to ION
- The Mongoose sites that are to be involved in the integration
- The planned mapping between the Mongoose sites and other BOD-enabled applications
- The message flow between the applications: what BODs need to flow, and in which direction?

Prerequisites

Before you attempt to take other steps to set up Mongoose-ION exchanges, make sure you have installed and configured the necessary hardware and software for all of these components:

- Infor Federation Services. See the *Infor Federation Services Installation Guide*.
- Infor ION Connect. See the *Infor ION Installation Guide*.
The server where you install ION must be able to connect to the servers where you install your Mongoose-based application and other BOD-enabled applications.
- Mongoose 9.00 or later. See the *Infor Mongoose Installation Guide*.

Setting up Mongoose for ION

This section provides the information and steps to set up Mongoose so that it can generate BODs and place them in the **Replication Document Outbox**, where ION Connect can retrieve them.

Setting up a configuration that matches your local site

You must specify a configuration name that *exactly* matches the local site name. This is required for non-transactional replication (replication of BODs). For example, if your local site name is “mylocalsite”, you must create a configuration named “mylocalsite”. If your configuration name already exactly matches the local site name, you can skip to the next section.

To set up a configuration that matches your local site:

- 1 On the Mongoose utility server, open the Configuration Manager utility and select the configuration you are currently using to access the local site.
- 2 Click **Copy** and specify a new configuration name that exactly matches the local site name as the target.
- 3 Stop and restart all Infor services.

Setting up the logical ID and BOD replication

To set up the logical ID and BOD replication in Mongoose:

Note: This procedure only needs to be done once, when replication is first being set up for outbound documents.

- 1 If multiple sites on a Mongoose intranet are to send or receive BODs, specify the "bootstrap" configurations on the **Replication** tab of the Service Configuration Manager utility on the Mongoose utility server:
 - In the **Replicator** group box, specify a **Configuration** that is to be the collection site for all outbound messages from sites on the intranet.
This site's **Replication Document Outbox** collects outbound BODs for all sites on the intranet.
 - In the **Inbound Bus** group box, specify the **Configuration** that is to be the collection site for all inbound messages from sites on the intranet.
This site's **Replication Document Inbox** collects inbound BODs for all sites on the intranet.

These two configurations can, and often do, point to the same site database. In many cases, the inbound and outbound sites are the same site.
- 2 On the **Intranets** form, create a new intranet for the Infor ION (bus), and specify these values:
 - **Intranet** – Specify a name to identify the intranet; for example: **InforBUS**
 - **Description** – Specify an appropriate description; for example: **Infor ION connection**
 - **External** – Select this check box.
 - **Transport** (on the **General** tab) – Specify **ESB**
 - **Tenant ID** – To use the same tenant ID for all sites on this intranet, specify it here.
You can also leave this field blank and specify the tenant ID on the local site records, or leave that field blank to use the default tenant ID value, **infor**.

The tenant ID can be a maximum of 22 characters and must match the tenant ID used in ION for other connecting applications.

For more information about how the tenant ID is used with ION, see the *ION Connect Administration Guide*.

3 On the **Sites** form, create a logical “site” for Infor ION:

- **Site** – Specify a name to identify this site as your ION site; for example: **InforBUS**
- **Site Name** and **Description** – Specify an appropriate site name and description.
- **Intranet Name** – Select the intranet you created in Step 2.
- **From Site** (on the **Site User Map** tab) – Specify the local site; for example: **mylocalsite**
- **User Name** – Specify the user ID to be used to send replication documents to the Mongoose **Replication Document Outbox** for ION to retrieve.

This user must already be set up on the **Users** form at the local site. We recommend that you specify **repl_user** here, if it is defined. Otherwise, specify a user with Full User editing permissions.

4 Still on the **Sites** form, configure the local site (for example, **mylocalsite**) to generate and/or receive BODs:

a Specify the **Message Bus Logical ID** for the site.

This identifies the Mongoose site to ION. Specify it as **infor.mg.site** where:

- **infor** is the tenant ID (does not have to be “infor”—see Step 2).
- **mg** (for Mongoose) is the application type.
- **site** is the name of the local site.

Example: **infor.mg.mylocalsite**

NOTE: The logical ID must be all lowercase. The logical ID entered here must match the application type and name defined in ION Connect for this Mongoose site.

b Optionally, specify a **Tenant ID** for the local site.

If no tenant ID is specified here or on the **Intranets** form, the tenant ID defaults to **infor**.

5 On the **Replication Rules** form, create a new rule with these specifications:

- **Source Site** – Specify the local site that you configured in Step 3.
- **Target Site** – Specify the ION site you created in Step 2.
- **Category** – Specify **ESB**.
- **Interval Type** – Specify **Immediate** or any other option *except* **Transactional**.

In most cases, you will want to specify **Immediate**, because that option causes replication to take place as soon as the specified replication operation is triggered. As is common, though, it depends on the need of your applications.

6 On the **Replication Management** form, click **Regenerate Replication Triggers**.

Setting up information that will flow to the other application

The information you set up in your Mongoose-based application (for example, customer, vendor, or item master records, or system parameters) depends on the application that it will exchange information with. For more information, see the integration guide for the application with which you want to exchange data.

Verifying that BODs are replicating

To verify that replication is set up properly and BODs are replicating:

- 1 On the Mongoose utility server, stop and start the Replication and Replication Queue Listener services.
- 2 In your Mongoose-based application, open a form whose information is being replicated to ION. For example, the information from a **Vendors** form information might be replicated.
- 3 Change a value and save the record.
In our example, you could change the vendor's address.
- 4 Check the **Replication Document Outbox** form to see if a BOD was generated.
In our example, a BOD named "SyncSupplierPartyMaster" should be generated.

Initial load: Running the Replication Document Manual Request Utility

To perform an initial load of Mongoose data into BODs that are placed in the **Replication Document Outbox**, run the Replication Document Manual Request Utility at each site. When running the utility, select **Initial Load**, to ensure that the BODs are loaded in the proper order.

When you use this utility to publish BOD data, the current date is used for all Status/EffectiveDateTime elements.

For more information about this utility, see the online help.

After the data is loaded initially, Mongoose sends updates automatically when users change certain data or when certain transactions occur in your Mongoose-based application. This all depends on how you have your ION connections set up and your document flow set up in ION. For more information see the appropriate documentation for Infor ION Connect.

Setting up ION for Mongoose

This section provides the information and steps to set up Infor ION Connect to import the Mongoose solution, configure the connection information for the Mongoose-based application, and activate the solution.

You must also configure the connection points for each application that will pass data to/from Mongoose through ION.

For additional information about using ION, see the *Infor ION Connect Administration Guide*.

About the Mongoose connection point

If you have integrated other products with your Mongoose-based application through ION, you probably already have a connection point for your Mongoose site(s) in ION. You can reuse this connection point for other ION integrations. However, you must make sure that the **Documents** tab for this connection point is set up to send and receive **ALL** appropriate documents for all integrations.

If you do not already have a connection point for Mongoose in ION, you will need to create one. For more information, and the procedure to do this, see the *Infor ION Connect Administration Guide*.

Activating the flow

To activate your solution in ION Desk:

- 1 On the **Document Flows** page, select the document flow between your Mongoose application and the other application, and click **Activate**.

If the activation succeeds, the system displays a success message.

- 2 Click **OK**.

Your changes are effective immediately.

If you receive an *error* message, the model cannot be activated. In this case, see the *Infor ION Connect Administration Guide* for assistance with activation errors.

If you receive a *warning* message, the model is still activated.

If you encounter problems during this setup, stop and restart the ION service.

Testing the integration

To verify that the integration has been set up successfully, create appropriate transactions in each application and ensure that the expected data is available in the other application.

Before starting the verification, make sure that these applications and service are running:

- Infor Mongoose
- The other BOD-enabled application
- IONService

Troubleshooting and maintenance

This section contains some common errors you might experience while working with BODs and ION in Mongoose-based applications.

Error in Replication Document Manual Request Utility – timeout

These are two errors you might receive during the generation of BODs through the Replication Document Manual Request utility:

```
Exception from load collection: Error processing an IDO request
(Protocol=Http, URL=http://jokerswild/IDORequestService/
RequestService.aspx):
The remote server returned an error: (500) Internal Server Error.
```

OR

```
The transaction has aborted.
Transaction Timeout
```

If you receive either of these errors during BOD generation, then transactions are taking longer to run than allowed by the timeout values set in your Mongoose-based application or other related applications and services.

For a list of timeout values that can be reset, see the chapter on improving performance in the *Infor Mongoose System Administration Guide*.

Data is not flowing properly between the systems

From the Infor ION Desk, select **Manage > Connectivity**. In that module, you can view:

- A deployment viewer that indicates status of messages
- A queue viewer that displays all the message queues from all the ION-enabled applications
- A list of error BODs
- A routing viewer that allows interactive searches to analyze the routing configurations

For more information, see the *Infor ION Administration Guide*.

Obsolete BODs – deleting

BODs in the **Replication Document Outbox** (COR_OUTBOX_xxx tables) are removed by Infor ION. ION Desk application polling properties control the removal of these BOD messages:

- If **Delete Processed Messages** is set to **True**, messages are deleted immediately after they are successfully processed by the ION run-time service.
- If **Delete Processed Messages** is set to **False**, messages are deleted if they:
 - Are older than the number of hours specified in the **Outbox Cleaner Expire Time** field
 - Have either been successfully sent or they do not have any assigned routes. The ION run-time service checks for expired messages when it is started, and then checks every hour.

By default, the second option is used. By not deleting the messages immediately, ION can monitor the outbox table and report the number of processed and unprocessed messages.

ION does not remove messages from the **Replication Document Inbox** (COR_INBOX_xxx) tables. If the inbox size becomes a problem, you can manually delete the records from the inbox table with a SQL Server script.

Drillbacks

“Drillback” refers to the ability to launch a Mongoose form in Infor Ming.le from a link based on the document ID for the relevant BOD. A link is defined by a “view” for each type of BOD noun (Sales Order, Item Master, etc.) and is then configured to launch a specific form for that noun. Not all nouns have a drillback view available.

Chapter 4: Using the Application Event System with IDOs

4

You can use Mongoose's Application Event System to send IDO request/responses using application framework events. For example, when an event occurs, it can trigger an update to a collection or execute an IDO method or stored procedure. This chapter covers the basic information to use the Application Event System to initiate IDO requests and responses, including descriptions of the event actions that can be used.

Event action types available for IDO integration

These are the action types available for IDO integration:

- **Dispatch IDO Request** – Dispatches an IDO request to a URL. This type of action opens a session, performs the request, and then closes the session.
You can use this action to request communication with the IDO Runtime Service at another “enterprise” (for example, one of your customers or vendors who also uses the same Mongoose-based application) or with another site in your own enterprise.
The URL for the receiving end must point to a Mongoose-based application.
Parse the returned result using the Transform XML event action type.
- **Execute IDO Request** – Executes an IDO request.
This is similar to the Dispatch IDO Request action, but it only goes to the current configuration's IDO runtime (that is, it acts only on the requester's own application database).
Parse the returned result using the Transform XML event action type.
- **Load Collection** – Loads rows of an IDO collection, up to the record cap.
- **Load IDO Row** – Retrieves the first matching row of data from an IDO collection.
There is a scenario showing how this works in the *Guide to the Application Event System*.
- **Update Collection** – Updates the data in a collection.
- **Call IDO Method** – Invokes an IDO method.
- **Call Database Method** – Directly calls a method in a database.
This is similar to a dynamic IDO (SP!).
- **Call Web Service** – Invokes and runs a particular .NET Web service.

For more information, see Chapter 5, "Using the .NET Web Service API".

- **Transform XML** – Performs an XSL transformation on the designated object.
Use this to parse elements from the result of a Dispatch IDO Request action or an Execute IDO Request action. You might also need it if, for example, the responding site is running a different version of the application (with a slightly different schema) than the requesting site.

Example: Opening a session and requesting data for a remote site

You can use the Application Event System to call into another site or to an external Mongoose environment and return a specific, discrete piece of information from the remote application database. This example from SyteLine returns the on-hand quantity of an item at another site.

You must create a new event that accepts two input parameters:

- **Item** specifies the item whose **On Hand** quantity you want to see.
- **Site** specifies the site from which the information is requested.

The event returns an output parameter called **OnHand** that contains the on-hand quantity as reported by the SLItems IDO for that site.

To set up and test this event, do this:

- 1 For added flexibility, define the site URLs as event global constants:

Name	Value
SiteA_URL	URL of SiteA's IDO Request Processor, for example: http://mySiteAUtilServer/IDORequestService/RequestService.aspx
SiteB_URL	URL of SiteB's IDO Request Processor, for example: http://mySiteBUtilServer/IDORequestService/RequestService.aspx

Later, you will use the pre-parser keyword TV() to select the appropriate one of these URLs, based on the site from which you want to get data. If both sites are on the same intranet, they can use the same URL.

- 2 Create a text file containing a parameterized IDO request to retrieve the on-hand quantity of an item:

```
<RequestHeader Type="LoadCollection">
  <InitiatorType />
  <InitiatorName />
  <SourceName />
  <TargetName />
  <RequestData>
    <LoadCollection Name="SL.SLItems" LoadCap="0">
      <PropertyList>
        <Item />
        <Description />
```

```

<RequestHeader Type="LoadCollection">
    <UnitCost />
    <CurUCost />
    <LotSize />
    <UnitWeight />
    <WeightUnits />
    <DerQtyOnHand />
</PropertyList>
<LoadType>FIRST</LoadType>
<RecordCap>-1</RecordCap>
<Filter>Item = {0}</Filter>
<OrderBy />
<PostQueryCmd />
</LoadCollection>
</RequestData>
</RequestHeader>

```

Notice the special syntax, **{0}**, that gives us the ability to substitute a different Item key for each run. Notice also that we include other properties that might be retrieved by other events, so we can reuse this request file in those events' handlers.

- 3 Name this file **Load1ItemInfo.xml**, and copy (or share) it to each utility server that will run the IDO Runtime Service. In this example, we place it on a common network folder which is mapped everywhere as V:\Scenarios.
- 4 In the **Event Handlers** form, create a handler where the **Event Name** is **GetRemoteOnHandQty** and the **Synchronous** check box is selected.
- 5 Click the **Event Actions** button and specify the following actions for the new event handler:

Sequence	Action Type	Parameters
1	Set Values	SETVARVALUES (SiteID=E (Site))
2	Dispatch IDO Request	URL (GC (TV (SiteID) _URL)) CONFIGNAME (E (Site)) SET (RV (IDO _Response) =RESULT) IDOREQUEST (SUBSTITUTE (FILECONTENTS ("V:\Scenarios\ Load1ItemInfo.xml"), FE (Item)))

Sequence	Action Type	Parameters
3	Transform XML	<pre>XML(V(IDO_Response)) XSLT("<?xml version='\"1.0\"'?> <xsl:stylesheet xmlns:xsl='\"http:// www.w3.org/1999/XSL/Transform\"' version='\"1.0\"'> <xsl:template match='\"/\"'> <xsl:value-of select='\"//P[last()]\"'/> </xsl:template> </xsl:stylesheet>") SET(RE(OnHand)=RESULT)</pre>

These actions are interpreted as follows:

- a Copy the value of the input parameter named **Site** into a variable named SiteID. We need this in order to use the TV() keyword in the next action.
- b Construct and dispatch an IDO request as follows:
 - Evaluate the variable named SiteID, append the string "_URL", and look up the result as the name of an event global constant. Use its value as the URL address to which to dispatch the request.
 - Signal the remote IDO Request Service to execute the request against its configuration named after the site. This configuration must exist there and point to that site's application database.
 - Read the contents of the file **V:\Scenarios\Load1ItemInfo.xml**, substituting the value of the input parameter named **Item** wherever {0} appears therein. Use the resulting value as the IDO request to be dispatched.
 - Put the IDO response into a new variable named **IDO_Response**. The value will look similar to this (varies depending on the item requested):

```
<ResponseHeader Type="LoadCollection">
  <InitiatorType />
  <InitiatorName />
  <SourceName />
  <TargetName />
  <ResponseData>
    <LoadCollection Name="SL.SLItems" LoadCap="0">
      <PropertyList>
        <Item />
        <Description />
        <UnitCost />
        <CurUCost />
        <LotSize />
        <UnitWeight />
        <WeightUnits />
        <DerQtyOnHand />
```



```

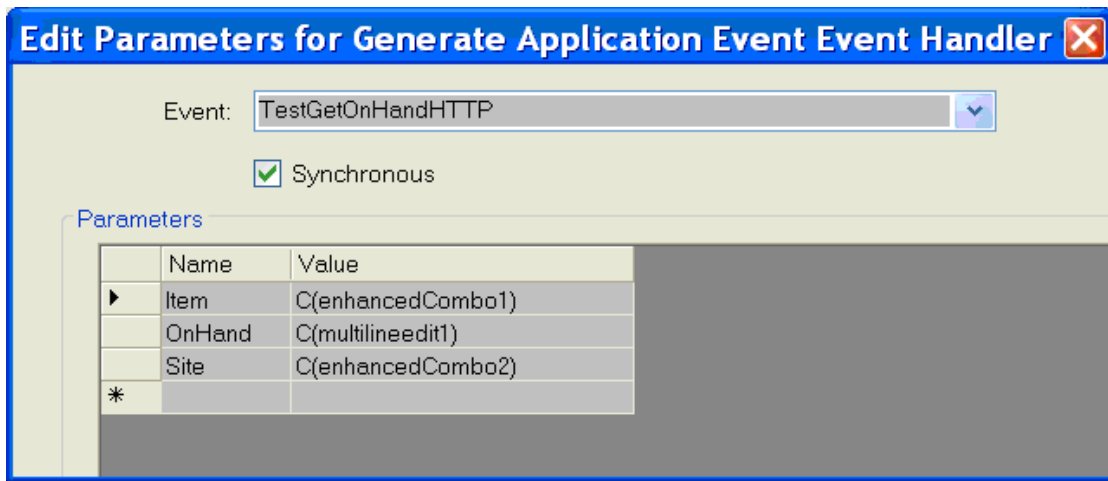
</PropertyList>
<LoadType>FIRST</LoadType>
<RecordCap>-1</RecordCap>
<Filter>Item = 'PT-70000'</Filter>
<OrderBy />
<PostQueryCmd />
<Items>
  <Item ID="PBT=[item] itm.DT=[20070618 08:30:06.250]
itm.ID=[c6cb54f1-258e-4ab6-ba0a-e182bc4d32c1]">
    <P>PT-70000</P>
    <P>Paint,Purple</P>
    <P>0.50000000</P>
    <P>0.50000000</P>
    <P>1.00000000</P>
    <P>1.00000</P>
    <P>LB</P>
    <P>279.00000000</P>
  </Item>
</Items>
</LoadCollection>
</ResponseData>
</ResponseHeader>

```

- c Perform an XSL transform on the contents of the variable named IDO_Response. The transform uses XPath to select all last children named P. (We know that **DerQtyOnHand** is the last property because we listed it last in the <PropertyList> of our IDO request. Instead of "[last()]" here, we could say the equivalent: "[8]") Put the result of the transform into an output parameter called **OnHand**.
- 6 To see the use of this event interactively, construct a new form like this:

The screenshot shows a web application window titled "Get Remote On Hand Quantity". Inside the window, there are two dropdown menus. The first is labeled "Site:" and has "Site A" selected. The second is labeled "Item:" and has "PT-70000" selected. Below these menus is a button labeled "Get Data". At the bottom of the window, there is a label "On Hand:" followed by a text input field that displays the value "279.00000000".

On the form's **Get Data** button, define a form event handler of response type Generate App Event with these parameters:



Edit Parameters for Generate Application Event Event Handler

Event:

☒ Synchronous

Parameters

	Name	Value
▶	Item	C(enhancedCombo1)
	OnHand	C(multilinedit1)
	Site	C(enhancedCombo2)
*		

- On your new form, the **Site** and **Item** drop-downs can easily be populated by inline and IDO collection list sources.

Additional Event Global Constants could be used for additional sites and/or external servers.

- Test the form: After selecting a site and item, click the **Get Data** button to dispatch the IDO request to the desired site's URL against the desired site's configuration. It should return and display the **On Hand** quantity for that item in that site's application database.

For more information

Details about creating and using events are found in the online help and in the *Guide to the Application Event System*.

It is possible to make requests over the Internet that load or update IDO collections, or that run IDO methods. The .NET Web service accesses Mongoose through an XML interface. This chapter covers ways to accomplish this, including:

- The kinds of API calls you can make to an IDO ("About .NET Web service API calls" on page 81)
- What must be done before you can use Web service API calls ("Prerequisite" on page 82)
- Prototype examples you can use to create API calls ("Creating API calls" on page 82)
- A working example (".NET Web service example" on page 88)
- How to use the .NET Web service to call a dynamic IDO ("Calling dynamic IDOs using .NET" on page 89)
- How to use the test utility to verify your .NET Web service connection ("Infor .NET Web service test utility" on page 90)

About .NET Web service API calls

The .NET Web service allows the following API calls to the IDO:

- **GetConfigurationNames** – Returns a list of valid configuration names
- **CreateSessionToken** – Validates your user ID password and configuration and returns a session token
- **LoadDataSet** – Returns data from an IDO
- **SaveDataSet** – Updates data through an IDO
- **CallMethod** – Calls an IDO method

You can also call dynamic IDOs using .NET, as described on page 89.

Prerequisite

Before you use .NET Web APIs to transfer data between your external interface and your Mongoose-based application, make sure the **Intranets** form includes the appropriate URL information for both the external application and your Mongoose-based application.

Creating API calls

Use the prototypes shown in this chapter as examples when creating your API calls.

All these prototype examples use C# code. Also, this code assumes you have added a reference to your project for the IDO .Net Web service, as described below.

In Visual Studio:

- 1 Right-click the **References** node in the solution explorer, or select **Add Service Reference** from the **Project** menu.
- 2 Enter the URL of your .NET Web Service:

`http://server_name/IDORequestService/IDOWebService.asmx`

- 3 Set **Namespace** to **IDOWebServiceReference**.
- 4 Click **OK**.

You can redirect calls at runtime (for instance, to a production server rather than a test server) by passing two parameters to the constructor. For example:

```
IDOWebServiceReference.DOWebServiceSoapClient idoWS =  
new IDOWebServiceReference.DOWebServiceSoapClient( "IDOWebServiceSoap",  
"http://other_server_name/IDORequestService/IDOWebService.asmx" );
```

Note: You can add a query parameter to the .NET Web Service URL to specify the configuration group, which applies to the GetConfigurationNames Web service API:

**`http://server_name/IDORequestService/IDOWebService.asmx?configgroup=
group-name`**

where *group-name* corresponds to a configuration group created with the Configuration Manager utility.

GetConfigurationNames

Returns a list of valid configuration names. This call only has access to configurations in the **Default** group.

Syntax

```
public string[] GetConfigurationNames()
```

Parameters

None

Return value

Returns all valid configuration names

Example

```
IDOWebServiceReference.DOWebServiceSoapClient webService =  
    new IDOWebServiceReference.DOWebServiceSoapClient( );  
string[] astrConfigNames;  
astrConfigNames = webService.GetConfigurationNames();
```

CreateSessionToken

Validates your user ID password and configuration and returns a session token.

When any of the other Web service methods (LoadDataSet, SaveDataSet, or CallMethod) are called, the session token that was returned from CreateSessionToken is a required parameter. This token is used to open a Mongoose session for each call to a Web service method, and the session is closed before the Web service method returns.

The user ID that is passed as a parameter to the CreateSessionToken must have the user module called "MGCoreAutomation" for the API call to work. This can be added using the **User Modules** form.

Note: In SyteLine, this user module is called "SyteLineAutomation."

Syntax

```
public string CreateSessionToken(string strUserId,  
    string strPswd,  
    string strConfig )
```

Parameters

Parameter	Description
strUserId	A valid User ID
strPswd	The password for the application user
strConfig	A valid configuration name used in the application

Return value

Returns a session token string, required by other Web service methods

Example

```
IDOWebServiceReference.DOWebServiceSoapClient webService =  
    new IDOWebServiceReference.DOWebServiceSoapClient( );  
SessionToken = webService.CreateSessionToken(  
    "sa",  
    "secret",  
    "SyteLine" );
```

LoadDataSet

Queries data through an IDO

Syntax

```
public DataSet LoadDataSet(string strSessionToken,  
    string strIDOName,  
    string strPropertyList,  
    string strFilter,  
    string strOrderBy,  
    string strPostQueryMethod,  
    int iRecordCap)
```

Parameters

Parameter	Description
strSessionToken	A session token string created by calling CreateSessionToken
strIDOName	The name of the IDO used to load the data set
strPropertyList	A comma-delimited list of property names to include in the data set

Parameter	Description
strFilter	A filter used to query records for the DataSet
strOrderBy	A comma delimited list of property names used to order the DataSet
strPostQueryMethod	The name of a method to run for each row in the result set after the query is completed
iRecordCap	Caps the number of records to returned by the query: 0 = unlimited -1 = default (200)

Return value

Returns a System.Data.DataSet containing the results of the query. The returned data can be filtered by any IDO-level filters. See the online help about IDO filters.

Example

```
IDOWebServiceReference.DOWebServiceSoapClient webService =
    new IDOWebServiceReference.DOWebServiceSoapClient( );
IdoDataSet = webService.LoadDataSet(
    SessionToken,
    "SL.SLItems",
    "Item, Description, UnitCost",
    "Item = N'FA-10000'",
    "", "", -1 );
```

SaveDataSet

Updates data through an IDO

Syntax

```
public DataSet SaveDataSet(string strSessionToken,
    DataSet UpdateDataSet,
    bool bRefreshAfterSave,
    string strCustomInsert,
    string strCustomUpdate,
    string strCustomDelete)
```

Parameters

Parameter	Description
strSessionToken	A session token string created by calling CreateSessionToken
UpdateDataSet	A DataSet containing rows to insert, update and delete
bRefreshAfterSave	If True , the UpdateDataSet rows are refreshed and returned to the caller
strCustomInsert	A comma-delimited list of methods and/or instructions which override the default save behavior
strCustomUpdate	A comma delimited list of methods and/or instructions which override the default save behavior
strCustomDelete	A comma delimited list of methods and/or instructions which override the default save behavior

Return value

Returns a dataset containing rows refreshed after the save, or null if bRefreshAfterSave is **False**.

Example

```
IDOWebServiceReference.DOWebServiceSoapClient webService =  
    new IDOWebServiceReference.DOWebServiceSoapClient( );  
string sessionToken;  
DataSet custDataSet;  
DataTable idoTable;  
DataRow newCust;  
sessionToken = webService.CreateSessionToken( "sa", "", "PWDEMO" );  
custDataSet = new DataSet( "SL.SLCustomers" );  
idoTable = custDataSet.Tables.Add( "IDO" );  
idoTable.Columns.Add( "CustNum", typeof(string) );  
idoTable.Columns.Add( "CurrCode", typeof(string) );  
idoTable.Columns.Add( "BankCode", typeof(string) );  
idoTable.Columns.Add( "_ItemId", typeof(string) );  
newCust = idoTable.Rows.Add( new object[] { "TEST123", "USD", "BK1", "" } );  
webService.SaveDataSet( sessionToken, custDataSet, true, "", "", "" );
```

CallMethod

Calls an IDO method

Syntax

```

public Object CallMethod(string strSessionToken,
    string strIDOName,
    string strMethodName,
    ref string strMethodParameters)

```

Parameters

Parameter	Description
strSessionToken	A session token string created by calling CreateSessionToken
strIDOName	The name of the IDO that publishes the method
strMethodName	The IDO method name to call
strMethodParameters	An XML formatted string containing the parameters to pass to the method. The strMethodParameters argument looks like this: <pre> <Parameters> <Parameter ByRef="Y N">value</Parameter> ...etc... </Parameters> </pre>

Return value

Returns the value that was returned to it by the method

Example

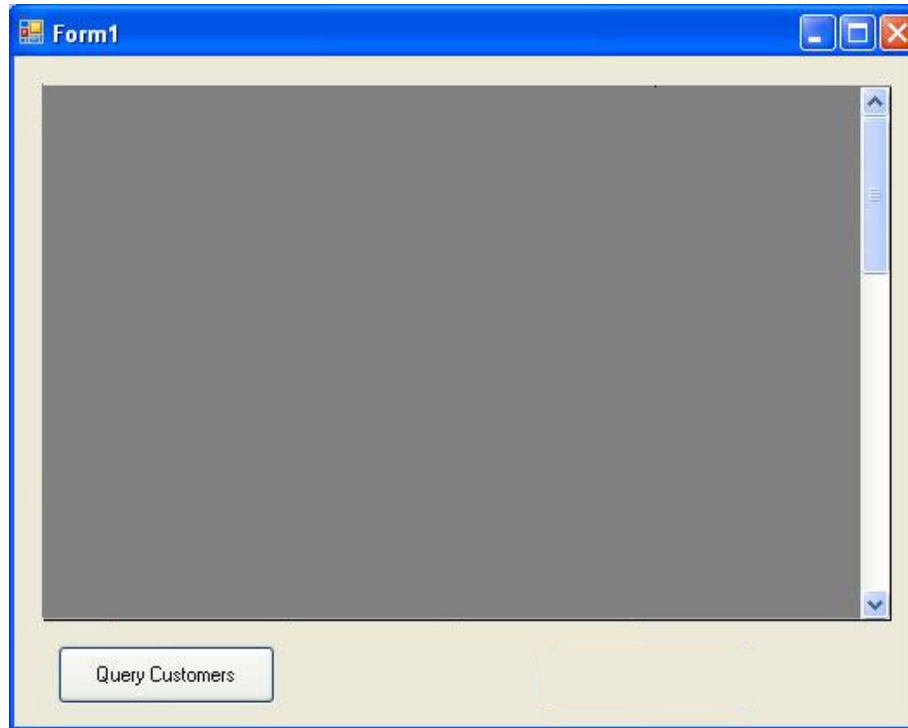
```

IDOWebServiceReference.DOWebServiceSoapClient webService =
    new IDOWebServiceReference.DOWebServiceSoapClient( );
object ReturnValue;
string strParameters = "<Parameters><Parameter
ByRef=\"Y\"/></Parameters>";
ReturnValue = webService.CallMethod(
    SessionToken,
    "SL.SLParms",
    "GetSessionIDSp",
    ref strParameters );

```

.NET Web service example

You could create an application form in C # with a display area and a **Query Customers** button, similar to this:



Below is the code behind the **Query Customers** button on the VB form.

```
String sessionToken;
DataSet idoDataSet;
IDOWebServiceReference.DOWebServiceSoapClient idoWS =
    new IDOWebServiceReference.DOWebServiceSoapClient( );
sessionToken = idoWS.CreateSessionToken(
    "user2",
    "",
    "SL800_Local" );
idoDataSet = idoWS.LoadDataSet(
    sessionToken,
    "SLCustomers",
    "Name, CustNum, CustSeq",
    "",
    "",
    "",
    0 );
dataGridView1.DataMember = idoDataSet.Tables[0].TableName;
```

```
dataGridView1.DataSource = idoDataSet;
```

When you click the **Query Customers** button, the form displays the SL customer name, number and sequence information, queried through the SLCustomers IDO:

	Name	CustNum	CustSeq
	Coordinated Bicy...	1	0
	Coordinated Bicy...	1	1
	Price Brothers De...	2	0
	Claymore Bicycles	3	0
	Moreclay Bicycles	3	1
	Precision Bicycles	3	2
	Gilmore Two Wh...	4	0
	Control Bicycles, ...	5	0
	Control Bicycles, ...	5	1
▶	Larry's Bicycles	6	0
	Travel Time Bicy...	7	0
	Time Travel Bicy...	7	1
	Ian's Bicycle Pro...	8	0

Query Customers

When setting this up, you must add a Web reference to the Mongoose WebService.

To do this:

- Right-click the References node in the solution explorer, or select **Add Service Reference** from the **Project** menu.
- Enter the URL of your .NET Web Service:
http://server_name/IDORequestService/IDOWebService.asmx
- Set **Namespace** to **IDOWebServiceReference**.
- Click **OK**.

Calling dynamic IDOs using .NET

IDO requests can directly query or update the SQL database tables calling dynamic IDOs. Through dynamic IDOs, you can also directly invoke stored procedures from the SQL database. To access a dynamic IDO, use the standard API call syntax, except:

- For collection requests, instead of the collection name, you specify the table name using the format **TABLE!table_name**.

The properties specified in the calls are actually the column names in the table.

- When invoking a method, instead of the IDO name, specify **SP!** and instead of the method name, specify the stored procedure name.
- When you open the session where you make these requests, specify a configuration. The configuration points to an application database. This is the database where the requests look for the tables and stored procedures.

Infor .NET Web service test utility

On the utility server, there is an executable file that provides a test client for the .NET Web service API. If you install the .NET Web service on your utility server and want to verify that it is installed and configured correctly without writing your own client, you can use this utility.

For more information, see the Infor .NET Web service test utility appendix in the installation guide for your system.

If your company uses SharePoint to share information among employees, the SharePoint pages can include charts and lists that display current Mongoose application data. For example, you can display a chart showing sales year-to-date by salesperson or a list of customers on credit hold. If a list is set up to allow it, you can also drill back to open the form that contains the data.

Mongoose SharePoint Web Parts retrieve and display on a SharePoint site the IDO data from a specific Mongoose configuration. You can download, deploy, and configure these Web Parts as described in this chapter, so that the Web Parts display specific Mongoose information. Administrators can include the configured Web Parts on shared pages, or they can export the configured Web Parts so that users can import them to their pages. Individual users can reconfigure each Web Part to point to a different configuration or to log in with a different user name.

This chapter covers the information and procedures needed to integrate Mongoose-based application data with SharePoint Web Parts, including:

- "About the IDO Web Parts" on page 91
- "Prerequisites" on page 92
- "Deploying the Web Parts in SharePoint" on page 92
- "Configuring the Web Parts to display Mongoose data in SharePoint" on page 96
- "Exporting Web Parts" on page 98
- "Importing Web Parts" on page 99
- "Drilling back to Mongoose application forms" on page 99

About the IDO Web Parts

These IDO Web Parts are available:

- **Infor Chart Web Part** – Displays a chart based on an Mongoose IDO. You can define some chart settings.
- **Infor List Web Part** – Displays a list based on an Mongoose IDO. This is similar to the collections of information you see on many forms.

These Web Parts are described in detail later in this section. For more information about what Web Parts are, and how to set up or use SharePoint, visit the MSDN Web site.

The deployment of these Web Parts also includes an administrative Web page where you configure the connection settings for the Web Parts.

Prerequisites

These applications must be installed:

- Mongoose 7.01 or later
- .NET 3.5 SP1
- SharePoint Foundation 2010, SharePoint Server 2010, Windows SharePoint Services 3.0, or Office SharePoint Server 2007
- Microsoft Chart Controls for Microsoft .NET Framework 3.5. The executable to install the chart controls (MSChart.exe) is included in the MGWebParts.zip file

Deploying the Web Parts in SharePoint

To load the Web Parts and then deploy and activate them in SharePoint:

- 1 Download the MGWebParts.zip file from the Infor Xtreme Support portal to the SharePoint server. The ZIP file contains the MGWebParts.wsp SharePoint deployment package and the Microsoft Chart Controls installation executable file.
- 2 Extract the contents of the MGWebParts.zip file to a folder on your SharePoint server.
- 3 On the Sharepoint server, open a new command prompt window with administrator privileges:
 - For Windows 2008 Server, right-click on the **Command Prompt** menu item and select **Run as administrator**.
 - For Windows 2003 Server, right-click on the **Command Prompt** menu item and select **Run as...** to display the **Run** dialog box, where you can choose to run with full administrative privileges.
- 4 In the command prompt window, change your current folder to the location where you unzipped the contents of the MGWebParts.zip file.
- 5 If you are upgrading from a previous version of the Infor Web Parts, retract and remove the old version:

Note: The path to **stsadm.exe** shown in the commands below is specific to SharePoint 3.0. For SharePoint 2010, the 12 must be changed to 14.

- a In the command prompt window, run this command to deactivate the feature:
"C:\Program Files\Common Files\Microsoft Shared\Web Server

Extensions\12\BIN\stsadm.exe" -o deactivatefeature -filename mgwebparts\feature.xml -url *SharepointServerName*

- b When that command completes, run this command to retract the solution:
"C:\Program Files\Common Files\Microsoft Shared\Web Server Extensions\12\BIN\stsadm.exe" -o retractsolution -name mgwebparts.wsp -immediate -url *SharepointServerName*
 - c When that command completes, run this command to delete the solution:
"C:\Program Files\Common Files\Microsoft Shared\Web Server Extensions\12\BIN\stsadm.exe" -o deletesolution -name mgwebparts.wsp
 - d Do not close the command prompt window, because you will use it later.
- 6 If the Microsoft Chart Controls for Microsoft .NET Framework 3.5 are not already installed, run MSChart.exe in the folder on the Sharepoint server where you extracted the contents of the zip file.
- 7 To add and deploy the solution and activate the feature in SharePoint:
- Note:** The path to stsadm.exe shown in the commands below is specific to SharePoint 3.0. For SharePoint 2010, the **12** must be changed to **14**.
- a In the command prompt window, run this command to add the SharePoint Web Parts solution to your SharePoint server:
"C:\Program Files\Common Files\Microsoft Shared\Web Server Extensions\12\BIN\stsadm.exe" -o addsolution -filename mgwebparts.wsp
 - b When that command completes, run this command to deploy the solution:
"C:\Program Files\Common Files\Microsoft Shared\Web Server Extensions\12\BIN\stsadm.exe" -o deploysolution -name mgwebparts.wsp -immediate -allowgacdeployment -url *SharepointServerName*
 - c When that command completes, run this command to activate the feature:
"C:\Program Files\Common Files\Microsoft Shared\Web Server Extensions\12\BIN\stsadm.exe" -o activatefeature -filename mgwebparts\feature.xml -url *SharepointServerName*

The Web Parts are now deployed and activated.

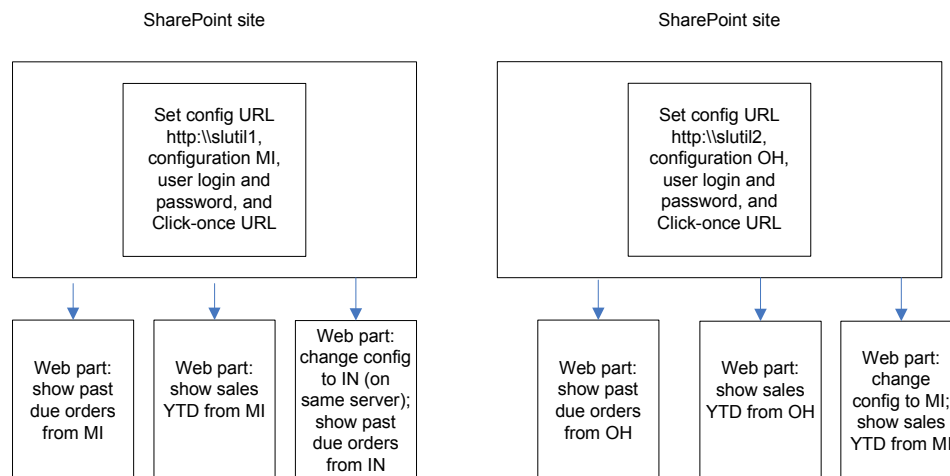
Configuring the connection to Mongoose at SharePoint sites

Any SharePoint user who displays information from a Mongoose-based application in a Web Part uses Mongoose configuration and login information to connect to the Mongoose-based application. The connection lasts only long enough to retrieve current data.

At a SharePoint site where the Web Parts are deployed, use the **Site Settings > Infor Web Part Configuration** page to configure the default connection settings for all Web Parts. The URL you specify at each site must be used by all Web Parts on that site; for individual Web Parts, users can override the configuration name, user ID, and password.

All Web Parts on a site connect to the same Mongoose configuration server, but individual Web Parts can override the configuration and user login set for a site. This allows you to compare data from different sites in side-by-side graphs on the same SharePoint page.

This example illustrates this:



To configure the connection for a site:

- 1 From a SharePoint site where the Web Parts are deployed, select **Site Actions > Site Settings**.
- 2 On the **Site Settings** page, under **Site Administration**, click **Infor web part configuration**.

Note: This page also displays the version number of the Web Parts.

- 3 Specify these settings:

- **Config Server URL** - Specify the URL to the configuration server, that is, the utility server where the configuration you want to access is defined. In the default value shown in the field, replace `<server-name>` with the appropriate server name, for example, **myserver.infor.com**. This URL cannot be overridden for individual Web Parts. This field is validated upon entry.
- **Config Name** - Select the Mongoose configuration to sign into. The drop-down list shows all available configurations on the configuration server you selected.
- **Change User ID/Password** - For individual Web Parts, select this field if you do not want to use the default login. If this field is not selected, the default user ID and password are disabled.
- **User ID** - Specify a user ID that exists in the specified configuration. This user must be assigned an MGCoreAutomation license. In certain cases, it is recommended that this user be a multi-session user. See "SharePoint licensing," below.

Note: In SyteLine, the MGCoreAutomation user module is called "SyteLineAutomation." Other applications might have other names for the "...Automation" user module.
- **Password** - Specify the password for that user.

- **ClickOnce Server URL** - Specify the URL to the utility server where the ClickOnce client code is installed. In the default value shown in the field, replace `<server-name>` with the appropriate server name, for example **myserver.infor.com**.

This URL is required only if you plan to allow drillbacks from the Web Parts. If a user selects the **View in Mongoose** (or comparable) menu option on the Web Part's **Edit** menu, and the user's computer does not already have the WinStudio client installed, the menu option uses this URL to install the thin ClickOnce client so the Web Part can open the configuration specified above.

- 4 Click **OK** to save the settings.

The system verifies the user ID and password for the selected configuration.

SharePoint licensing

When you configure the Infor Web Parts, you must specify a valid (Mongoose-based application) user ID, password, and configuration for the application site. This user must be granted the *AppAutomation* license module.

Note: In most Mongoose-based applications, this user module is named "MGCoreAutomation." In SyteLine, this user module is called "SyteLineAutomation." Other applications might have other names for the "*AppAutomation*" user module.

All Infor Web Parts on a site can share a single default set of login credentials (user ID, password, and configuration), or you can override the user ID, password, and configuration on specific Web Parts. For moderate use, a single dedicated user ID is sufficient because connections to the Mongoose application are created and released as data is queried for the various Web Parts.

However, if your SharePoint site is heavily used, and if most users will use the default login rather than overriding it with their own user IDs, then you might need a multi-session user for the default login, to support higher levels of concurrent use. In that case, specify the **AppInternal** user module that is provided with each application's license. **AppInternal** is a multi-session user. When you grant the **AppAutomation** license module to the **AppInternal** user, it consumes the number of sessions defined for **AppInternal**. To see how many sessions are defined for **AppInternal**, use the **License Management** form.

Adding the Web Parts to a SharePoint page

To add Web Parts to a page in SharePoint:

- 1 To create a page, select **Site Actions > Create**, or to edit an existing page, select **Site Actions > Edit**.

If you are creating a page, click **Web Part Page** and specify the filename and layout template to use.

- 2 Click **Add a Web Part** in the area of the page where you want to add the Web Part.

- 3 Select one of the Infor Web Parts from the list. The Web Part displays on the page with a note that the IDO name must be specified.
- 4 Click the **Edit** down-arrow on the Web Part and select **Modify Shared Web Part**.
- 5 Configure the Web Part settings as described in the following sections.

After you configure a Web Part, you can export it so that other users can import it with all the configuration settings intact.

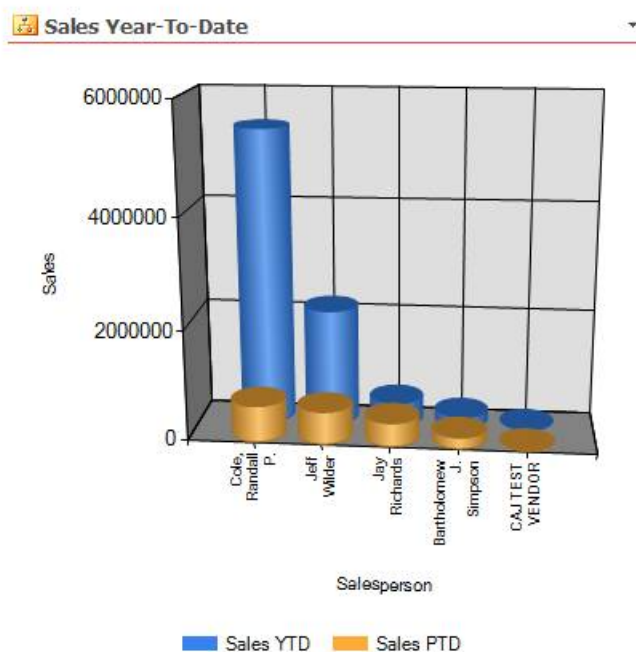
Configuring the Web Parts to display Mongoose data in SharePoint

To configure a Web Part, first add it as described above, and select **Modify Shared Web Part** from the **Edit** menu on the Web Part.

If you are not sure which IDO to use in a Web Part, display a form in your Mongoose-based application that includes the information you want to include in the Web Part. In Design Mode on the form, view the form properties. The **Collections** tab lists the IDOs being used in the form.

Infor Chart Web Part

Use this Web Part to display a chart based on IDO data like the one below:



This example displays two series. The legend displays the translated series property names.

In the **Edit** dialog box, use these fields to set up the Web Part:

- **Override Config Settings** - Select this check box if you want this Web Part to use a different configuration or login and password than the default that was set at the site. When you select this check box, additional fields display. For information about these fields, see “Configuring the connection to Mongoose at SharePoint sites” on page 93.
- **IDO Name** - Select the IDO that has the properties you want to display.
- **X-Axis Property** - Select the property from the IDO that you want to display on the X-axis (horizontal axis) in the chart. For example, in the chart above, the salesperson name is displayed on the X-axis. Note that some properties might not make sense to display in a chart format.
- **Series Properties** - Select one or more properties to display in the chart. To add properties to the chart, click the >> button. To remove them from the chart, click the << button. Note that series properties are the actual values that are charted and must be numeric.
- **Filter** - If you want to limit the information, for example, to a certain set of customer numbers, specify the filter information here. Any valid SQL WHERE clause syntax is allowed. Here are some examples:

```
CoNum like 'ABC%'
QtyOnHand < 20 AND Whse = 'OH'
```

- **Order By** - Specify a comma-delimited list of properties that specify how the results are to be sorted.
- **Chart Title** - Specify the title that is to appear over the chart.
- **X-Axis Title** - Specify the title for the X-Axis (horizontal axis).
- **Y-Axis** - Specify the title for the Y-Axis (vertical axis).
- **Chart Height** - Specify the chart height, in pixels, or set a fixed height that fits the space defined for the Web Part.
- **Chart Width** - Specify the chart width, in pixels, or set a fixed width that fits the space defined for the Web Part.
- **Chart Type** - Select the desired chart type. Note that some chart types are inappropriate for some types of data.
- **Background Color** and **Secondary Background Color** - Use these properties to create a color gradient in the chart.

Other self-explanatory chart options are available.

Infor List Web Part

Use this Web Part to display a tabular list based on a Mongoose IDO like the one below:

Credit Hold				
	Name	Customer	Ship To	City
Select	Control Bicycles, Inc.	5	0	Rockford
Select	Larry's Bicycles	6	0	Seattle
Select	Travel Time Bicycles	7	0	Bloomington
Select	test	A000017	0	

Use the fields as described here:

- **Override Config Settings** - Select this check box if you want this Web Part to use a different configuration or login and password than the default that was set at the site. When you select this check box, additional fields display. For information about these fields, see “Configuring the connection to Mongoose at SharePoint sites” on page 93.
- **IDO Name** - Select the IDO that has the properties you want to display.
- **IDO Properties** - Select the properties from the IDO that you want to include in the listing. To add properties to the list, click the >> button. To remove them from the list, click the << button.
- **IDO Filter** - If you want to limit the information, for example, to a certain set of customer numbers, specify the filter information here. Any valid SQL where clause syntax is allowed. Here are some examples:

```
CoNum like 'ABC%'
```

```
QtyOnHand < 20 AND Whse = 'OH'
```

- **IDO Order By** - Specify a comma-delimited list of properties that specify how the results should be sorted.
- **Grouping Column** - Use this field to group information in the displayed list. For example, if the list contains a **State** column, you can set the **Grouping Column** to **State** and the grid displays data grouped by state.
- **Drillback Form Name** - If you want users to be able to drill down on a row in the list to open a form showing the currently selected data, select the form name here (for example, **AccountingPeriods**). For additional information about drillback, see “Drilling back to Mongoose application forms” on page 99.
- **Drillback Filter** - If you specified a drillback form name, and you want the form to open filtered on certain data, specify the filter expression here. This filter expression is similar to WinStudio collection filter expressions, except the only valid substitution keyword is **FP (filter property)**. Only properties that are displayed in the Web Part can be included in the drillback filter.

For example, if your Web Part displays data from the GroupNames IDO and includes the GroupId property, you can enable drillback to the **Group Names** form by specifying **Group Names** for the drillback form, and this drillback filter expression:

```
GroupId = FP(GroupId)
```

For additional information about drillback, see “Drilling back to Mongoose application forms” on page 99.

Appearance, layout, advanced

These sections are common to all Web Parts and are not specific to Mongoose Web Parts.

Exporting Web Parts

After a Web Part is configured, you can export the Web Part so that other users can import it rather than having to set up their own configurations.

To export a configured Web Part:

- 1 Select **Export** on the **Edit** menu of the Web Part.
- 2 In the dialog box, browse to the area where you want to store the Web Part file and specify a filename. The file extension should be **.webpart**.
- 3 Click **Save**.

Importing Web Parts

To import a Web Part that is already configured:

- 1 Edit the page where you want to import the part.
- 2 Select **Add Web Part**.
- 3 From the **Add Web Part** dialog box, click the **Advanced Web Part gallery and options** link.
- 4 In the advanced **Add Web Parts** editor, click the down-arrow next to **Browse** and select **Import**.
- 5 Click **Browse...** and select the **.webpart** file to import.
- 6 Click **Upload**.
- 7 Click **Import**.

Note: The Mongoose configuration name, user, and password are not exported along with the Web Part. When you import a Web Part, this information defaults to the global site-level settings.

Drilling back to Mongoose application forms

If a Web Part is configured to allow drillback, then when you select a row by clicking the **Select** link, you can select **View In App** (where *App* is the name of your Mongoose-based application) from the **Edit** menu. This option opens the WinStudio client on your computer and displays the form and property data that is linked to that row in the Web Part. If you are not currently logged in to your Mongoose-based application, a login dialog box displays.

In order for drillback to work, the ClickOnce Server URL must be specified in the default connection information (see “Configuring the connection to Mongoose at SharePoint sites” on page 93). A drillback form name must be specified during configuration of the Web Part. A drillback filter can also be specified during the configuration; then, when the user selects the **View In App** menu option, the form opens with a filter created using the drillback filter expression and the property values from the selected row. If no filter is specified, the form opens unfiltered.

Chapter 7: Directly Accessing the Database Programmatically

7

You can programmatically access the application database directly (without going through the IDO layer), using a development tool such as VB.NET or C#.

The IDO layer usually handles the session management for connections. Thus, if your program is bypassing the IDO layer, you must use the procedures in this chapter to perform session management.

InitSessionContextSp

Before you execute any code on a database connection that might modify any application tables or call any application stored procedure, you must first initialize your session by calling this procedure.

```
EXEC dbo.InitSessionContextSp
  @ContextName = 'caller-context'
  , @SessionID = @MySessionID OUTPUT
  , @Site = 'site'
```

Parameter	Description
@ContextName	This parameter (<i>caller-context</i>) can be anything. We recommend that you put something meaningful in this value so that it is obvious what process initiated the call.
@SessionID	If this parameter is null, the InitSessionContextSp procedure creates a new session ID and returns it as an output parameter. If this parameter is not null, the value that is passed is used for the session ID. You can use the same @SessionID to initialize multiple connections. All connections that are initialized using the same @SessionID value will share a common set of session variables. However, you should never use the same session ID for two connections that are running in different transactions simultaneously.

Parameter	Description
@Site	This parameter value (<i>site</i>) should be the site value found in the Configuration Manager for the configuration that would have been used to login using Winstudio. This parameter is used to get the correct information from views over <code>_mst</code> tables; for example, the parms view over the <code>parms_mst</code> table.

Database settings

The IDO layer makes these settings whenever a connection is opened. If your code will access any application code, your application or process should also make these settings whenever it opens a connection:

Setting	Description
SET ANSI_WARNINGS, ANSI_NULLS, ANSI_PADDING, XACT_ABORT, CONCAT_NULL_YIELDS_NULL, ARITHABORT, QUOTED_IDENTIFIER NOCOUNT ON	ANSI_WARNINGS, ANSI_NULLS, ANSI_PADDING, ARITHABORT, CONCAT_NULL_YIELDS_NULL, and QUOTED_IDENTIFIER are required for computed column indexes to work. NOCOUNT ON prevents every single operation in a trigger or stored procedure from returning a count to the client of how many rows were affected, which would be a performance problem given the large number of operations performed in the database. XACT_ABORT is required for distributed transaction support.
SET NUMERIC_ROUNDABORT OFF	NUMERIC_ROUNDABORT must be off for computed column indexes.

Your application or process must also make this setting whenever a connection is opened:

Setting	Description
SET TRANSACTION ISOLATION LEVEL [READ COMMITTED READ UNCOMMITTED]	Set the TRANSACTION ISOLATION LEVEL to the level required by the application that is about to run. The level is not reset during connection pooling, so each new user of a connection must specifically set the isolation level.

CloseSessionContextSp

When your application or process is finished (assuming you did not roll back the entire transaction), you should call this procedure to clean up any session variables that might have been created.

```
EXEC CloseSessionContextSp @SessionID = @MySessionID
```


Parameter	Description
@SessionID	This should be the session ID that was specified in the call to InitSessionContextSp

Index

Symbols

.NET

- CallMethod 86
- CreateSessionToken 83
- Dynamic IDOs 89
- GetConfigurationNames 82
- LoadDataSet 84
- SaveDataSet 85
- Test Utility 90
- Web Service API 81
- Web service example 88

A

- ActiveScriptingOnly tag 32
- AdditionalFailureInformation tag 35
- AllowCloseExistingSessions tag 35
- ANSI_NULLS 102
- ANSI_PADDING 102
- ANSI_WARNINGS 102
- App Metadata Sync utility 23
- App Metadata Transport utility 23
- AppDB class 15
- ApplicationName tag 35
- AppName tag 31
- AppURL tag 31
- ARITHABORT 102
- AuditingEnabled tag 36
- Authorizations 21

B

- BaseHelpFile tag 32
- BinaryDataFormat tag 41

BODs

- BOD nouns 64
- BOD verbs 64
- defined and described 64
- verifying replication of 68

- BoolFalse tag 41

- BoolTrue tag 41

- business object documents, see BODs

C

- CallMethod, .NET API call 86
- Case tag 42
- ClassNotesExist attribute 40
- Client class library 14
- CloseSession, in XML documents 63
- CloseSessionContextSp 102
- Closing session 102
- ColSecurityPrimaryOnly tag 33
- ColumnType tag 41
- CompanyName tag 31
- CONCAT_NULL_YIELDS_NULL 102
- Config tag 31
- ConfigGroup tag 31
- ConfigName tag 35
- configserver.aspx 29
- Configuration Manager 23
- Configuration Manager utility 31
- connection point in ION 69
- Copyright tag 31
- CreateSessionToken, .NET API call 83
- Cursor tag 46

D

Database settings during programmatic database connection 102

Database, accessing programmatically 101

Database, used for virtual IDOs 65, 90

DataType tag 40

Date format, when passing to IDOs 64

DateFormat tag 42

DaysUntilPasswordExpires tag 36

DeadlockRetry tag 35

DefaultIMECharset tag 41

DefaultValue tag 41

DefFormFont 32

DeleteExistingPrompt tag 32

DeleteNewPrompt tag 32

Direct access to database 101

DisableInvalidValuePrompt tag 33

DisplayDecimal Position tag 42

Distinct tag 47

DomainIDOName tag 41

DomainUserName tag 35

drillbacks in ION integrations 71

Dynamic IDO

Definition 16

XML documents 64

E

EditLevel tag 36

ent 9

Error response for XML UpdateCollection 57

Extension classes 14

F

Filter tag 47

FormCLSID tag 31

FormFileExtension tag 31

FormsURL tag 31

FromXML method 13

G

GetConfigurationNames, .NET API call 82

GetConfigurations, in XML documents 26, 29

GetPropertyInfo tag 40

GetPropertyInfo, in XML documents 38

H

HelpBasePath tag 31

HelpContentstIndexMenuText tag 32

HorzComponentSpacing tag 32

Html tag 41

HTTP POST command 26

HTTPS protocol 26

I

IconFilename tag 31

ID attribute, UpdateCollection 59

Identity tag 41

IDO

.NET class libraries 13

Command interface 14

definition 9

Extension classes 14

Passing dates to 64

Runtime Development Server 12

IDOProperties tag 40

IDOProtocol class library 13

IDORequest 11

IDORequest class 13

IDORequestEnvelope 11, 13

IDOWebService example 88

IncludeClassNotesFlag attribute 40

Infor Chart Web Part 91

Infor List Web Part 91

information needed for integration with ION 65

InitiatorName, in XML documents 29

InitiatorType, in XML documents 28

InitSessionContextSp 101

InputMask tag 41

InputMaskPromptChar tag 41

InsertPostedToWorkflow attribute 40

Invoke, in XML documents 61

ION

- activating the flow 69
- BODs, use of 64
- connection point for Mongoose 69
- defined and described 63
- drillbacks 71
- exchanges with Mongoose 64
- information needed for integration 65
- prerequisites for integration 65
- setup for Mongoose 65
- setup in ION 69
- testing the integration 69
- troubleshooting and maintenance 70

Item tag 47

J

Justify tag 42

L

Label tag 41

LabelStringID tag 41

LanguageID tag 35

License tag 35

Licensing, SharePoint 95

LinkBy tag 41, 47, 53

LoadCollection tag 46

LoadCollection, in XML documents 44

LoadDataSet, .NET API call 84

LoadType tag 46

LoginOverrideProgID tag 31

LoginResult tag 36

M

MachineName tag 35

Method, definition 9

MGWebParts.zip file 92

Microsoft Chart Controls 93

Middleware permissions 21

Mongoose.IDO.Client class library 13

MSChart.exe 93

N

NOCOUNT 102

NumDigitsInGroup tag 42

NUMERIC_ROUNDABORT 102

O

Object authorizations for user 21

ObjectMenuName tag 32

OpenSession, in XML documents 34

OrderBy tag 48

P

Password tag 35

Payload 11

Permissions 21

Posting XML documents 26

PostQueryCmd tag 48

prerequisites for ION integration 65

PrimaryCacheUsesFormCaption tag 32

PrimaryGroupName tag 36

ProductVersion tag 35

ProgIDRootAugment tag 32

Program access to database 101

Property tag 40

Property, definition 9

PropertyList tag 47

Protocol class library 13

Protocol version elements in XML documents 28

Q

Query Customers example 88

QUOTED_IDENTIFIER 102

R

ReadMode tag 46

ReadOnly tag 41

RecordCap tag 47

RegionalSettings tag 35

Replication Document Manual Request Utility 68

ReportViewerExecutable tag 32

RequestHeader elements in XML documents 27

RequestService.ASP 26

RequestType elements in XML documents 28

Required tag 41

ResponseHeader elements in XML documents 27

RightJustifyNewLabels tag 33

RORRecord tag 41

RSDAO COM library 16

S

SaveDataSet, .NET API call 85

Secure transmissions 26

ServerDate tag 36

Session

 Closing 102

 Initialization 101

 Token 83

SessionID elements in XML document 28

SharePoint

 Licensing 95

 Web parts 91

SourceName, in XML documents 29

SP! format 17, 64

SplashFilename tag 31

Standard IDO, definition 17

StartupMethods tag 36

Subcollection tag 41

T

TABLE! format 64

TargetName, in XML documents 29

Token for session 83

ToXML method 13

TRANSACTION ISOLATION LEVEL 102

troubleshooting ION integrations 70

U

UpdateCollection

 Error response 57

 in XML documents 51

 Item ID example 59

 Nested Insert example 55

 UseKeys example 60

UseHtmlHelp tag 32

UseKeys attribute, UpdateCollection 60

User middleware permissions 21

UserId tag 35

V

ValidationErrorIndicators tag 33

VBAFilename tag 31

verifying replication of BODs 68

Version tag 31

VertComponentSpacing tag 32

Virtual IDOs 64

W

WallpaperDisplayOption tag 32

WallpaperFilename tag 32

Warning tag 41

Web parts 91

Web service example 88

Wire Tap 12

Workstation tag 35

X

XACT_ABORT 102

XML

CloseSession 63

Dynamic IDOs 64

GetConfigurations 26, 29

GetPropertyInfo 38

InitiatorName 29

InitiatorType 28

Invoke 61

LoadCollection 44

OpenSession 34

Posting documents 26

ProtocolVersion 28

Request and response documents 25

RequestHeader 27

RequestType 28

ResponseHeader 27

SessionID 28

SourceName 29

TargetName 29

UpdateCollection 51

XPVisualStyle tag 33